

Implementasi MapReduce untuk Identifikasi Anomali pada Log Akses

Big Data Analitik



Muhammad Farrel Akbar (22/492806/TK/53947)
Muhammad Aqil Fauzaan (22/494733/TK/54301)
Syaifullah Hilmi Ma'arij (22/497775/TK/54568)
Tsaniya Khamal Khasanah (22/503817/TK/55074)
Brandon Rafael Lovelyno (22/500359/TK/54847)
Muhamad Daffa Azfa Rabbani (22/503970/TK/55101)

Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Daftar Isi

BAB I.....	5
1. PENDAHULUAN.....	5
1.1. Latar Belakang.....	5
1.2. Rumusan Masalah.....	5
1.3. Tujuan Penelitian.....	6
1.4. Manfaat Penelitian.....	6
1.5. Batasan Masalah.....	6
BAB II.....	7
2. PROSES MAP DAN REDUCE.....	7
2.1. Tujuan Pemrosesan Data Menggunakan MapReduce.....	7
2.2. Alur Umum Proses MapReduce.....	7
2.3. Tahapan Map Function.....	8
2.3.1. Input Mapper.....	8
2.3.2. Proses Parsing Log dan Pembentukan Key-Value Pair.....	9
2.4. Tahapan Shuffle & Sort.....	10
2.5. Tahapan Reduce Function.....	11
2.5.1 Proses Agregasi Nilai.....	11
2.5.2 Output Tahap Reduce.....	11
2.6. Algoritma / Pseudocode Map.....	12
2.7. Algoritma / Pseudocode Reduce.....	12
2.8. Diagram Alur Proses MapReduce.....	13
2.8.1 Diagram Alur Proses Map.....	13
2.8.2 Diagram Alur Proses Reduce.....	13
BAB III.....	14
3. IMPLEMENTASI HADOOP.....	14
3.1. Lingkungan Komputasi yang Digunakan.....	14
3.1.1. Spesifikasi Perangkat Keras & Perangkat Lunak.....	14
Spesifikasi Perangkat Keras.....	14
Spesifikasi Perangkat Lunak.....	14
3.1.2. Mode Deployment.....	15
3.2. Instalasi dan Konfigurasi Hadoop.....	16
3.2.1. Download dan Ekstraksi.....	16
3.2.2. Environment Variables.....	17
3.2.3. Configuration Files.....	19
c) mapred-site.xml.....	21
3.2.4. SSH Configuration.....	23
3.2.5. Format HDFS.....	24
3.2.6. Start Hadoop Services.....	25
3.3. Persiapan Dataset pada HDFS.....	26

3.3.1. Dataset Information.....	26
3.3.2. Upload to HDFS.....	27
3.4. Implementasi Program MapReduce.....	28
3.4.1. Project Structure.....	28
3.4.2. Mapper Implementation.....	28
3.4.3. Reducer Implementation.....	29
3.4.4. Driver Implementation.....	30
3.5. Kompilasi dan Packaging.....	31
3.5.1. Kompilasi.....	31
3.5.2. JAR Packaging.....	32
3.6. Eksekusi Job MapReduce.....	33
3.6.1. Submit Job.....	33
3.6.2. Job Progress.....	34
3.6.3. Job Counters.....	35
3.7. Monitoring dan Verifikasi.....	36
3.7.1. YARN Web UI.....	36
3.7.2. Output Verification.....	38
3.7.3. Sample Results.....	39
3.7.4. Total Unique IPs.....	40
Kesimpulan Tahap Monitoring & Verifikasi.....	40
BAB IV.....	42
4. IMPLEMENTASI SISTEM.....	42
4.1. Pembahasan.....	42
BAB V.....	45
5. KESIMPULAN.....	45
Daftar Pustaka.....	46

BAB I

1. PENDAHULUAN

1.1. Latar Belakang

Perkembangan teknologi internet dan layanan berbasis web menyebabkan meningkatnya jumlah aktivitas pengguna yang mengakses sebuah website. setiap akses tersebut dicatat oleh web server dalam bentuk web server access log, yang berisi informasi penting seperti alamat IP client, waktu akses, metode HTTP, serta sumber daya yang diminta. Seiring meningkatnya trafik pengguna, ukuran data access log yang dihasilkan juga semakin besar dan dapat mencapai skala Big Data.

Pada studi kasus ini dataset web access log yang digunakan memiliki ukuran sekitar 3.4 GB dan berisi lebih dari 10 juta baris record. Ukuran data yang besar ini menyulitkan proses analisis apabila dilakukan dengan metode konvensional atau menggunakan satu mesin dengan pemrosesan single-threaded, karena akan membutuhkan waktu yang lama dan sumber daya yang besar. Selain itu, di dalam data web access log terdapat potensi aktivitas anomali, seperti alamat IP yang melakukan request secara berlebihan dalam waktu singkat, yang dapat mengindikasikan aktivitas bot, crawler, maupun potensi ancaman keamanan terhadap server.

Untuk mengatasi permasalahan tersebut, diperlukan suatu pendekatan yang mampu memproses data dalam jumlah besar secara efisien dan terstruktur. Salah satu solusi yang dapat digunakan adalah Apache Hadoop dengan model pemrograman MapReduce, yang dirancang untuk melakukan pemrosesan data secara paralel dan terdistribusi. Dengan memanfaatkan MapReduce, data access log dapat dipecah menjadi bagian-bagian kecil, diproses secara bersamaan oleh banyak mapper, dan digabungkan kembali pada tahap reduce untuk memperoleh hasil analisis yang diinginkan.

1.2. Rumusan Masalah

Berdasarkan permasalahan yang telah dijelaskan sebelumnya, beberapa hal yang menjadi fokus pembahasan adalah sebagai berikut:

1. Bagaimana cara memproses data web server access log berukuran besar secara efisien?
2. Bagaimana menerapkan model pemrograman MapReduce untuk menghitung frekuensi akses setiap IP Address?
3. Bagaimana proses ekstraksi dan agregasi data IP Address dilakukan dalam tahapan MapReduce?
4. Bagaimana menghasilkan data keluaran yang merepresentasikan jumlah akses setiap IP Address sebagai dasar analisis anomali?

1.3. Tujuan Penelitian

Tujuan dari studi kasus ini adalah:

1. Menganalisis web server access log berukuran besar menggunakan pendekatan Big Data.
2. Menerapkan model pemrograman MapReduce untuk menghitung frekuensi akses setiap IP Address.
3. Menjelaskan alur pemrosesan data melalui tahapan Map, Shuffle & Sort, dan Reduce.
4. Menghasilkan output berupa pasangan (IP Address, jumlah akses) yang dapat digunakan untuk analisis lanjutan.

1.4. Manfaat Penelitian

Manfaat yang diperoleh dari pembahasan ini antara lain:

1. Memberikan pemahaman mengenai penerapan MapReduce dalam pemrosesan data web server access log.
2. Membantu proses analisis trafik website melalui perhitungan frekuensi akses IP Address.
3. Menjadi dasar dalam mendeteksi IP Address yang berpotensi melakukan aktivitas anomali.
4. Menjadi referensi pembelajaran dalam penerapan konsep Big Data dan Hadoop MapReduce.

1.5. Batasan Masalah

Supaya penelitian ini lebih terfokus dan terarah, maka batasan masalah yang ditetapkan adalah:

1. Dataset yang digunakan berupa web server access log dari Kaggle dengan ukuran sekitar 3.4 GB.
2. Analisis difokuskan pada perhitungan frekuensi akses berdasarkan IP Address.
3. Proses analisis dilakukan menggunakan model pemrograman MapReduce.
4. Penelitian ini tidak membahas metode mitigasi keamanan atau penanganan serangan jaringan secara langsung.
5. Analisis lanjutan terhadap IP anomali dibatasi pada hasil perhitungan jumlah akses.

BAB II

2. PROSES MAP DAN REDUCE

2.1. Tujuan Pemrosesan Data Menggunakan MapReduce

MapReduce digunakan untuk menganalisis Web Server Access Log dengan tujuan utama menghitung frekuensi akses setiap IP Address ke sebuah website. Analisis ini penting untuk berbagai keperluan seperti memahami pola traffic pengunjung, mendeteksi aktivitas bot atau crawler, serta mengidentifikasi potensi ancaman keamanan seperti serangan DDoS atau brute force.

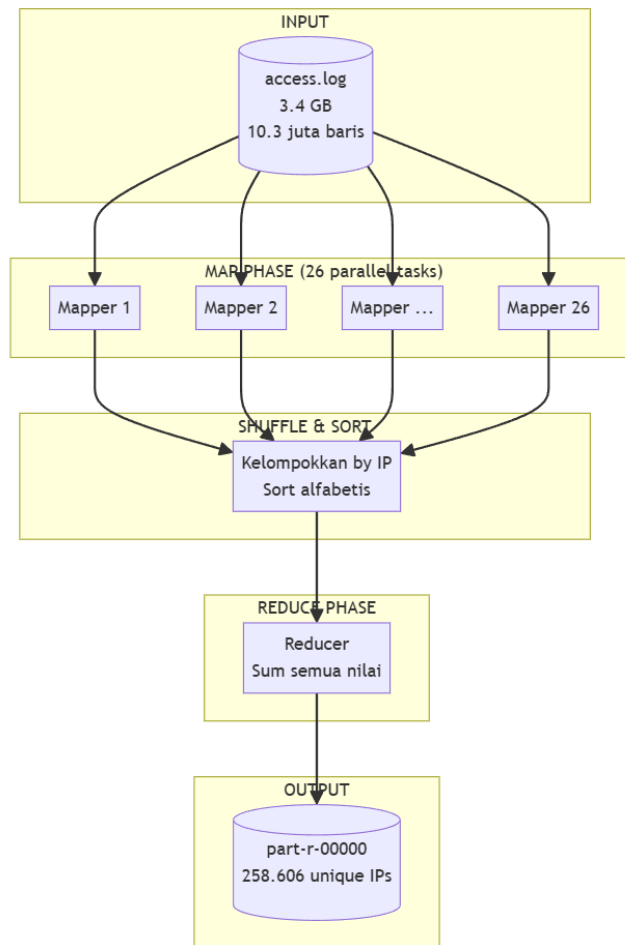
Dataset yang digunakan merupakan access log dari Kaggle dengan ukuran 3.4 GB dan berisi lebih dari 10 juta baris record. Dengan karakteristik Big Data seperti ini, pemrosesan secara konvensional menggunakan single-threaded akan memakan waktu sangat lama dan tidak efisien. MapReduce hadir sebagai solusi karena memungkinkan pemrosesan data secara paralel, dimana data dipecah menjadi bagian-bagian kecil dan diproses secara bersamaan oleh multiple mapper. Selain itu, MapReduce juga menawarkan skalabilitas yang baik untuk menangani data yang jauh lebih besar serta fault tolerance yang memastikan proses tetap berjalan meskipun terjadi kegagalan pada salah satu node.

2.2. Alur Umum Proses MapReduce

Secara garis besar, proses MapReduce untuk kasus IP Count Analysis terdiri dari tiga tahap utama yaitu **Map, Shuffle & Sort, dan Reduce**. Pada tahap pertama, file access.log yang tersimpan di HDFS dibaca dan dipecah menjadi 26 bagian (input splits) yang kemudian diproses secara paralel oleh 26 mapper. Setiap mapper bertugas mengekstrak IP Address dari setiap baris log dan menghasilkan pasangan key-value berupa (IP, 1).

Setelah semua mapper selesai bekerja, output dari mapper akan memasuki fase Shuffle & Sort. Pada fase ini, semua pasangan (IP, 1) yang tersebar dari berbagai mapper akan dikelompokkan berdasarkan IP Address yang sama dan diurutkan secara alfabetis. Hasil dari fase ini adalah kumpulan pasangan di mana setiap IP memiliki list nilai, misalnya (IP, [1, 1, 1, ...]).

Tahap terakhir adalah Reduce, di mana Reducer menerima hasil dari Shuffle & Sort dan melakukan agregasi dengan menjumlahkan semua nilai 1 untuk setiap IP. Output akhir dari proses ini adalah pasangan (IP, total_count) yang menunjukkan berapa kali setiap IP mengakses website. Dari 10.365.152 baris input, proses ini menghasilkan 258.606 unique IP Address dengan masing-masing count-nya.



2.3. Tahapan Map Function

2.3.1. Input Mapper

Data input yang diproses oleh Mapper berupa file access log dalam format Apache Combined Log Format. File ini merupakan catatan setiap request yang masuk ke web server, di mana setiap baris merepresentasikan satu kali akses.

```

54.36.149.41 - - [22/Jan/2019:03:56:14 +0330] "GET /filter/27|13%20%D9%85%DA%AF%D8%A7%D9%BE%DB%8C%DA%A9%D8%B3%I
31.56.96.51 - - [22/Jan/2019:03:56:16 +0330] "GET /image/60844/productModel/200x200 HTTP/1.1" 200 5667 "https:,
31.56.96.51 - - [22/Jan/2019:03:56:16 +0330] "GET /image/61474/productModel/200x200 HTTP/1.1" 200 5379 "https:,
40.77.167.129 - - [22/Jan/2019:03:56:17 +0330] "GET /image/14925/productModel/100x100 HTTP/1.1" 200 1696 "-" "I
91.99.72.15 - - [22/Jan/2019:03:56:17 +0330] "GET /product/31893/62100/%D8%B3%D8%B4%D9%88%D8%A7%D8%B1~%D8%AE%D
40.77.167.129 - - [22/Jan/2019:03:56:17 +0330] "GET /image/23488/productModel/150x150 HTTP/1.1" 200 2654 "-" "I
40.77.167.129 - - [22/Jan/2019:03:56:18 +0330] "GET /image/45437/productModel/150x150 HTTP/1.1" 200 3688 "-" "I
40.77.167.129 - - [22/Jan/2019:03:56:18 +0330] "GET /image/576/article/100x100 HTTP/1.1" 200 14776 "-" "Mozill
66.249.66.194 - - [22/Jan/2019:03:56:18 +0330] "GET /filter/b41,b665,c150%7C%D8%A8%D8%AE%D8%A7%D8%B1%D9%BE%D8%
40.77.167.129 - - [22/Jan/2019:03:56:18 +0330] "GET /image/57710/productModel/100x100 HTTP/1.1" 200 1695 "-" "I
  
```

Setiap baris dalam access log memiliki struktur yang terdiri dari beberapa field yang dipisahkan oleh spasi. Field pertama adalah IP Address dari client yang melakukan request, diikuti oleh remote log name, user identity, timestamp, HTTP request yang berisi method, path, dan protocol, kemudian HTTP status code, response size dalam bytes, dan terakhir adalah referer serta user agent. Berikut adalah contoh satu baris access log.

```
54.36.149.41 - - [22/Jan/2019:03:56:14 +0330] "GET /filter/27|13" HTTP/1.1" 200 30577 "-"  
"Mozilla/5.0"
```

Pada contoh di atas, IP Address yang akan diekstrak adalah **54.36.149.41** yang terletak di posisi paling awal sebelum spasi pertama. Dalam konteks Hadoop MapReduce, file `access.log` dibaca secara line-by-line oleh framework. Setiap baris dikirim ke fungsi mapper dalam bentuk pasangan key-value, di mana key berupa byte offset (posisi baris dalam file) dengan tipe `LongWritable`, dan value berupa isi baris log tersebut dengan tipe `Text`.

2.3.2. Proses Parsing Log dan Pembentukan Key-Value Pair

Proses parsing dalam mapper bertujuan untuk mengekstrak IP Address dari setiap baris log. Karena IP Address selalu berada di posisi pertama dan dipisahkan oleh spasi dari field berikutnya, maka algoritma parsing yang digunakan cukup sederhana namun efektif.

Ketika mapper menerima sebuah baris log, langkah pertama yang dilakukan adalah mengkonversi value dari tipe `Text` menjadi `String` agar dapat dimanipulasi. Selanjutnya, mapper mencari posisi spasi pertama dalam string tersebut menggunakan `indexOf(' ')`. Posisi spasi ini menjadi delimiter yang memisahkan IP Address dari sisa baris log.

Setelah posisi spasi pertama ditemukan, mapper melakukan validasi untuk memastikan bahwa spasi memang ditemukan (nilai `firstSpace` lebih besar dari 0). Jika valid, mapper mengekstrak IP Address dengan mengambil substring dari index 0 hingga posisi spasi pertama. Hasil ekstraksi inilah yang menjadi output key dari mapper.



Untuk setiap IP Address yang berhasil diekstrak, mapper mengeluarkan (emit) pasangan key-value berupa (IP, 1). Key adalah IP Address dalam bentuk `Text`, sedangkan value adalah konstanta 1 dalam bentuk `IntWritable`. Nilai 1 ini merepresentasikan satu kali kemunculan atau satu kali akses dari IP tersebut. Penggunaan nilai konstan 1 memungkinkan proses agregasi yang sederhana di tahap Reduce, di mana semua nilai 1 untuk IP yang sama akan dijumlahkan untuk mendapatkan total frekuensi akses.

Untuk memberikan gambaran konkret tentang bagaimana mapper bekerja, berikut adalah contoh eksekusi dengan tiga baris sample input:

Input:

Baris 1: 54.36.149.41 - - [22/Jan/2019:03:56:14 +0330] "GET /filter" 200

Baris 2: 31.56.96.51 - - [22/Jan/2019:03:56:16 +0330] "GET /image" 200

Baris 3: 54.36.149.41 - - [22/Jan/2019:03:56:17 +0330] "GET /product" 200

Proses dan Output:

Baris 1: `indexOf(' ')=12 → substring(0,12) → emit("54.36.149.41", 1)`

Baris 2: `indexOf(' ')=11 → substring(0,11) → emit("31.56.96.51", 1)`

Baris 3: `indexOf(' ')=12 → substring(0,12) → emit("54.36.149.41", 1)`

Dapat dilihat bahwa IP `54.36.149.41` muncul dua kali dengan masing-masing value 1. Agregasi untuk menghitung total akan dilakukan di tahap Reduce, bukan di tahap Map. Hal ini sesuai dengan prinsip MapReduce di mana mapper hanya bertugas melakukan transformasi dan ekstraksi data, sedangkan reducer bertugas melakukan agregasi. Berdasarkan hasil eksekusi job MapReduce yang dilakukan, tercatat bahwa Map input records dan Map output records sama-sama berjumlah 10.365.152. Kesamaan angka ini menunjukkan bahwa setiap baris input menghasilkan tepat satu output pasangan (IP, 1), yang berarti tidak ada baris yang di-skip dan semua baris memiliki format yang valid dengan IP Address yang dapat diekstrak.

2.4. Tahapan Shuffle & Sort

Setelah tahap Map selesai dan seluruh pasangan key-value dihasilkan melalui pemanggilan `context.write(key, value)`, data hasil keluaran mapper tidak langsung diteruskan ke reducer. Output dari setiap mapper terlebih dahulu disimpan pada buffer di memori, kemudian secara bertahap ditulis ke disk lokal ketika buffer tersebut penuh. Pada proses ini, Hadoop secara otomatis melakukan pengurutan (sort) berdasarkan key serta membagi data ke dalam partisi-partisi sesuai dengan jumlah reducer yang ditentukan. Dengan demikian, setiap mapper akan menghasilkan beberapa file output lokal yang telah terurut dan siap untuk diproses lebih lanjut.

Tahap berikutnya adalah Shuffle, yaitu proses pemindahan data antar node yang melibatkan komunikasi jaringan. Pada fase ini, reducer secara aktif menarik (pull) data hasil mapper dari node-node tempat mapper dijalankan. Setiap reducer hanya mengambil partisi data yang menjadi tanggung jawabnya, sehingga seluruh pasangan key-value dengan key yang sama akan dikirim ke reducer yang sama, meskipun berasal dari mapper yang berbeda. Proses inilah yang menyebabkan Shuffle menjadi fase paling mahal secara komputasi dan jaringan dalam MapReduce, karena melibatkan transfer data dalam jumlah besar antar node.

Input:

Mapper 1:

```
emit("54.36.149.41", 1)
```

```
emit("31.56.96.51", 1)
```

Mapper 2:

```
emit("54.36.149.41", 1)
```

Output Shuffle & Sort:

Input Reducer (setelah Shuffle & Sort):

Key: "54.36.149.41" → Values: [1, 1]

Key: "31.56.96.51" → Values: [1]

2.5. Tahapan Reduce Function

2.5.1 Proses Agregasi Nilai

Setelah seluruh proses Shuffle & Sort selesai, reducer menerima input berupa satu key beserta kumpulan nilai (values) yang terkait dalam bentuk iterable. Pada tahap ini, fungsi `reduce()` mulai dijalankan untuk setiap key secara terpisah. Proses agregasi dilakukan dengan melakukan iterasi terhadap seluruh nilai pada iterable tersebut dan menjumlahkannya untuk memperoleh total kemunculan key yang bersangkutan. Dengan mekanisme ini, reducer mampu menghitung jumlah akses untuk setiap alamat IP secara menyeluruh, tanpa bergantung pada urutan data asli maupun asal mapper yang menghasilkan pasangan key–value tersebut.

2.5.2 Output Tahap Reduce

Hasil dari proses agregasi pada tahap Reduce berupa pasangan key–value baru yang merepresentasikan total jumlah kemunculan setiap key. Berdasarkan input reducer yang diterima setelah tahap Shuffle & Sort, keluaran reducer dapat ditunjukkan sebagai berikut.

Output Reduce:

("54.36.149.41", 2)

("31.56.96.51", 1)

Output ini menunjukkan bahwa alamat IP 54.36.149.41 tercatat melakukan dua kali akses, sedangkan alamat IP 31.56.96.51 tercatat satu kali akses. Pasangan key–value hasil Reduce ini selanjutnya disimpan ke dalam HDFS sebagai hasil akhir dari proses MapReduce dan dapat digunakan untuk analisis lanjutan, seperti pengurutan IP dengan jumlah akses tertinggi.

2.6. Algoritma / Pseudocode Map

Berikut adalah algoritma mapper yang digunakan untuk mengekstrak IP Address dari web server access log

```
PROCEDURE Map(key, value, context)
  // Input: key (byte offset), value (baris log)
  // Output: (IP Address, 1)

  line ← value.toString()
  firstSpace ← line.indexOf(' ')

  IF firstSpace > 0 THEN
    ip ← line.substring(0, firstSpace) // Extract IP
    context.write(ip, 1) // Emit key-value pair
  END IF
END PROCEDURE
```

Algoritma di atas diimplementasikan dalam bahasa Java sebagai class `IPCountMapper` yang meng-extend class `Mapper` dari Hadoop MapReduce framework. Class ini menerima input dengan tipe `<LongWritable, Text>` dan menghasilkan output dengan tipe `<Text, IntWritable>`. Method `map()` akan dipanggil secara otomatis oleh Hadoop framework untuk setiap baris input, dan hasil emit akan dikumpulkan untuk dikirim ke tahap Shuffle & Sort.

2.7. Algoritma / Pseudocode Reduce

```
REDUCE(key, list_of_values):
  total ← 0
  for each value in list_of_values do
    total ← total + value
  emit(key, total)
```

Pada tahap Reduce, fungsi `reduce()` menerima satu key beserta kumpulan nilai (values) yang telah dikelompokkan pada tahap Shuffle & Sort. Pseudocode di atas menunjukkan proses agregasi sederhana berupa penjumlahan seluruh nilai dalam iterable untuk setiap key guna memperoleh total kemunculan yang bersangkutan. Proses ini dijalankan secara terpisah untuk setiap key dan tidak bergantung pada urutan data maupun sumber mapper yang menghasilkan nilai tersebut. Dengan pendekatan ini, tahap Reduce berperan sebagai komponen utama dalam melakukan agregasi data dan menghasilkan keluaran akhir berupa pasangan key-value yang merepresentasikan hasil perhitungan global untuk setiap alamat IP.

2.8. Diagram Alur Proses MapReduce

2.8.1 Diagram Alur Proses Map

Diagram berikut menggambarkan alur detail proses yang terjadi di dalam fungsi Mapper, mulai dari penerimaan input hingga menghasilkan output pasangan key-value:

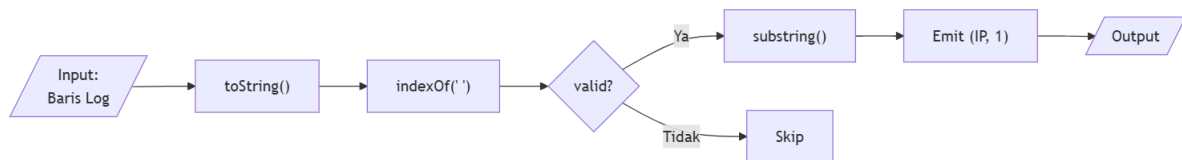
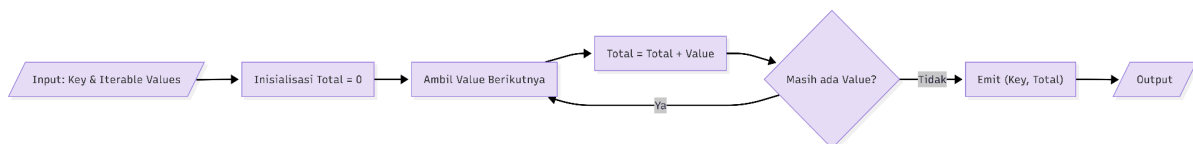


Diagram di atas menunjukkan alur proses mapper secara horizontal dari kiri ke kanan. Proses dimulai ketika mapper menerima input berupa satu baris log dari HDFS. Baris tersebut pertama-tama dikonversi menjadi String. Selanjutnya, mapper mencari posisi spasi pertama dengan `indexOf(' ')` untuk menentukan batas IP Address. Jika posisi spasi valid (lebih besar dari 0), maka IP Address diekstrak menggunakan `substring()` dan di-emit sebagai pasangan (IP, 1) ke output. Namun jika tidak valid, baris tersebut akan di-skip dan mapper melanjutkan ke baris berikutnya. IP Address yang sama dapat muncul berkali-kali dengan value 1, dan penggabungan (agregasi) akan dilakukan pada tahap Reduce selanjutnya.

2.8.2 Diagram Alur Proses Reduce



Flowchart Reduce menggambarkan proses agregasi data yang dilakukan setelah seluruh pasangan key–value dikelompokkan pada tahap Shuffle & Sort. Reducer menerima satu key beserta kumpulan nilai yang terkait dalam bentuk iterable, kemudian menginisialisasi sebuah variabel untuk menyimpan hasil perhitungan. Selanjutnya, reducer melakukan iterasi terhadap setiap nilai dan menjumlahkannya hingga seluruh nilai diproses. Setelah proses agregasi selesai, reducer menghasilkan keluaran berupa pasangan key–value baru yang merepresentasikan total kemunculan setiap alamat IP, yang kemudian disimpan sebagai hasil akhir dari tahap Reduce.

BAB III

3. IMPLEMENTASI HADOOP

3.1. Lingkungan Komputasi yang Digunakan

Proses implementasi dan pengujian Hadoop pada penelitian ini dilakukan menggunakan lingkungan komputasi berbasis **WSL2 (Windows Subsystem for Linux 2)** yang berjalan di atas sistem operasi Windows 11. WSL2 dipilih karena menyediakan lingkungan Linux yang kompatibel dengan Hadoop, memiliki performa mendekati native Linux, serta tidak memerlukan alokasi sumber daya terpisah sebagaimana pada mesin virtual konvensional. Pendekatan ini termasuk kategori **on-premise environment**, karena seluruh proses eksekusi dijalankan langsung pada perangkat fisik milik pengguna.

3.1.1. Spesifikasi Perangkat Keras & Perangkat Lunak

Spesifikasi Perangkat Keras

Perangkat yang digunakan memiliki spesifikasi sebagai berikut:

- Processor: Intel Core i7 (16 cores)
- RAM: 16 GB DDR5
- Storage: 512 GB NVMe SSD
- GPU: NVIDIA RTX 4060 8 GB
- OS Host: Windows 11
- OS Environment: WSL2 Ubuntu 24.04

Konfigurasi prosesor multi-core mendukung eksekusi parallel pada proses MapReduce, sementara kapasitas RAM 16 GB memungkinkan pengalokasian memori yang memadai untuk layanan Hadoop serta proses pendukung lainnya. Media penyimpanan NVMe SSD dipilih karena memiliki kecepatan I/O yang tinggi, yang diperlukan untuk proses ini pembacaan dan penulisan blok data Hadoop. GPU tidak digunakan secara langsung pada eksekusi MapReduce karena pemrosesan utamanya berbasis CPU.

Spesifikasi Perangkat Lunak

Lingkungan perangkat lunak yang digunakan terdiri atas:

- Hadoop versi 3.3.6
- Java OpenJDK 11
- Python 3.12.3 (digunakan untuk analisis hasil dan post-processing)

Pemilihan Hadoop 3.3.6 dan Java 11 dilakukan karena keduanya merupakan versi stabil dan saling kompatibel untuk eksekusi layanan Hadoop berbasis YARN. Python digunakan sebagai pendukung proses analisis lanjutan terhadap output MapReduce.

3.1.2. Mode Deployment

Implementasi Hadoop dijalankan dalam **mode pseudo-distributed**, di mana seluruh komponen Hadoop (NameNode, DataNode, ResourceManager, dan NodeManager) berjalan pada satu perangkat, namun tetap dipisahkan dalam proses yang berbeda sebagaimana pada lingkungan terdistribusi. Mode ini dipilih karena:

- dataset yang diproses masih berada pada skala yang dapat ditangani satu mesin,
- memudahkan proses konfigurasi dan debugging selama pengembangan,
- lebih efisien dari sisi sumber daya dibandingkan konfigurasi cluster multi-node.

Screenshot spesifikasi perangkat yang digunakan:

```
Swap: 2.0Gi 0 2.0Gi
farrel@LAPTOP-0ROBEU3T:~$ lscpu
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 39 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 24
On-line CPU(s) list: 0-23
Vendor ID: GenuineIntel
Model name: 13th Gen Intel(R) Core(TM) i7-13700HX
CPU family: 6
Model: 191
Thread(s) per core: 2
Core(s) per socket: 12
Socket(s): 1
Stepping: 2
BogoMIPS: 4608.00
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr p
```

```
farrel@LAPTOP-0ROBEU3T:~$ nvidia-smi
Fri Dec 26 11:07:24 2025
+-----+
| NVIDIA-SMI 590.44.01          Driver Version: 591.44      CUDA Vers
ion: 13.1 |
+-----+
+-----+
| GPU Name                     Persistence-M | Bus-Id      Disp.A | Volatil
e Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap |      Memory-Usage | GPU-Uti
l  Compute M. |
| MIG M. |
+-----+-----+
| 0  NVIDIA GeForce RTX 4060 ...    On   | 00000000:01:00:0 Off |
| N/A   56C    P0              21W / 120W | 0MiB / 8188MiB | 0%
| Default |
| N/A |
+-----+-----+
+-----+
+-----+
| Processes: |
| GPU GI   CI          PID    Type    Process name
| GPU Memory |
| ID       ID
| Usage
+-----+
| No running processes found
+-----+
```

3.2. Instalasi dan Konfigurasi Hadoop

Proses instalasi Hadoop dilakukan pada lingkungan Ubuntu 24.04 di dalam WSL2. Instalasi dilakukan secara manual melalui proses pengunduhan paket Hadoop resmi dari situs Apache, ekstraksi berkas instalasi, serta konfigurasi variabel lingkungan agar seluruh layanan Hadoop dapat dijalankan secara otomatis melalui terminal.

3.2.1. Download dan Ekstraksi

Tahap pertama yaitu melakukan pengunduhan paket Hadoop versi 3.3.6 dalam format arsip `.tar.gz` dari repository resmi Apache. Setelah berkas berhasil diunduh, arsip diekstraksi sehingga menghasilkan direktori instalasi Hadoop. Direktori hasil ekstraksi kemudian dilakukan penamaan ulang menjadi `hadoop` agar struktur path lebih sederhana dan mudah digunakan pada proses konfigurasi berikutnya.

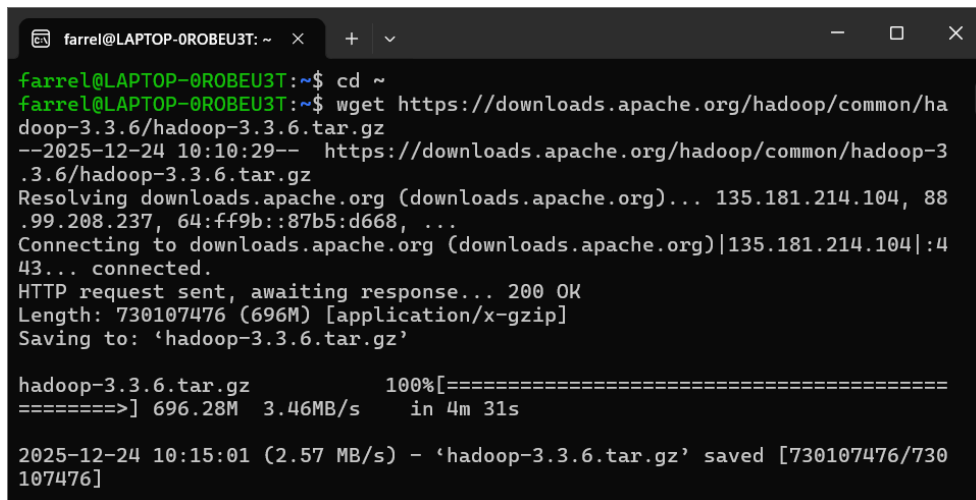
Langkah ini bertujuan untuk:

- memastikan berkas instalasi berasal dari sumber resmi dan valid,
- mendapatkan struktur direktori Hadoop yang lengkap (bin, sbin, lib, dan konfigurasi),
- menyederhanakan pengelolaan path ketika melakukan pembaruan versi di kemudian hari.

Command:

```
wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz
tar -xzf hadoop-3.3.6.tar.gz
mv hadoop-3.3.6 hadoop
```

Screenshot:



```
farrel@LAPTOP-0ROBEU3T: ~  
farrel@LAPTOP-0ROBEU3T:~$ cd ~  
farrel@LAPTOP-0ROBEU3T:~$ wget https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz  
--2025-12-24 10:10:29-- https://downloads.apache.org/hadoop/common/hadoop-3.3.6/hadoop-3.3.6.tar.gz  
Resolving downloads.apache.org (downloads.apache.org)... 135.181.214.104, 88.99.208.237, 64:ff9b::87b5:d668, ...  
Connecting to downloads.apache.org (downloads.apache.org)|135.181.214.104|:443... connected.  
HTTP request sent, awaiting response... 200 OK  
Length: 730107476 (696M) [application/x-gzip]  
Saving to: 'hadoop-3.3.6.tar.gz'  
  
hadoop-3.3.6.tar.gz      100%[=====]  
=====>] 696.28M  3.46MB/s   in 4m 31s  
  
2025-12-24 10:15:01 (2.57 MB/s) - 'hadoop-3.3.6.tar.gz' saved [730107476/730107476]
```

```

farrel@LAPTOP-0ROBEU3T:~$ ls -lh hadoop-3.3.6.tar.gz
-rw-r--r-- 1 farrel farrel 697M Jun 26 2023 hadoop-3.3.6.tar.gz
farrel@LAPTOP-0ROBEU3T:~$ tar -xzf hadoop-3.3.6.tar.gz
farrel@LAPTOP-0ROBEU3T:~$ mv hadoop-3.3.6 hadoop
farrel@LAPTOP-0ROBEU3T:~$ ls -lh hadoop/
total 112K
-rw-r--r-- 1 farrel farrel 24K Jun 14 2023 LICENSE-binary
-rw-r--r-- 1 farrel farrel 15K Jun 10 2023 LICENSE.txt
-rw-r--r-- 1 farrel farrel 29K Jun 10 2023 NOTICE-binary
-rw-r--r-- 1 farrel farrel 1.6K Jun 10 2023 NOTICE.txt
-rw-r--r-- 1 farrel farrel 175 Jun 10 2023 README.txt
drwxr-xr-x 2 farrel farrel 4.0K Jun 18 2023 bin
drwxr-xr-x 3 farrel farrel 4.0K Jun 18 2023 etc
drwxr-xr-x 2 farrel farrel 4.0K Jun 18 2023 include
drwxr-xr-x 3 farrel farrel 4.0K Jun 18 2023 lib
drwxr-xr-x 4 farrel farrel 4.0K Jun 18 2023 libexec
drwxr-xr-x 2 farrel farrel 4.0K Jun 18 2023 licenses-binary
drwxr-xr-x 3 farrel farrel 4.0K Jun 18 2023 sbin
drwxr-xr-x 4 farrel farrel 4.0K Jun 18 2023 share

```

Verifikasi Instalasi

Verifikasi dilakukan dengan menjalankan perintah `hadoop version`. Output yang dihasilkan menampilkan informasi versi Hadoop beserta detail kompilasi, yang menandakan bahwa:

- direktori instalasi telah dikenali oleh sistem,
- variabel lingkungan berhasil dikonfigurasi,
- dependensi Java telah terdeteksi dengan benar.

Apabila perintah tidak dikenali atau sistem menampilkan pesan kesalahan terkait `JAVA_HOME`, maka hal tersebut mengindikasikan bahwa path variabel lingkungan belum terkonfigurasi secara benar.

```

source ~/.bashrc
hadoop version

```

```

farrel@LAPTOP-0ROBEU3T:~$ nano ~/.bashrc
farrel@LAPTOP-0ROBEU3T:~$ source ~/.bashrc
farrel@LAPTOP-0ROBEU3T:~$ hadoop version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da
9266a4f88195058f08fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1

```

3.2.2. Environment Variables

Konfigurasi variabel lingkungan dilakukan melalui berkas `~/.bashrc`, sehingga seluruh pengaturan dapat dijalankan secara otomatis ketika terminal WSL dibuka. Variabel yang ditambahkan meliputi:

- `HADOOP_HOME`: mendefinisikan lokasi direktori instalasi Hadoop,
- `PATH`: menambahkan direktori `bin` dan `sbin` agar perintah Hadoop dapat dijalankan dari direktori mana pun,

- **JAVA_HOME**: mendefinisikan lokasi instalasi OpenJDK 11 sebagai runtime Java yang digunakan Hadoop.

Penggunaan perintah **export** memastikan variabel tersebut dapat diakses oleh seluruh proses turunan (child process) Hadoop. Setelah perubahan disimpan, konfigurasi diaktifkan menggunakan perintah **source ~/.bashrc** sehingga tidak memerlukan restart terminal.

File: nano ~/.bashrc

Konfigurasi dengan menambahkan kode dalam bagian bash:

```
export HADOOP_HOME=~/.hadoop
export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```

Screenshot:



```
GNU nano 7.2 /home/farrel/.bashrc
alias l='ls -CF'

# Add an "alert" alias for long running commands.  Use like so:
#   sleep 10; alert
alias alert='notify-send --urgency=low -i "${?} = 0" && echo terminal ||>'

# Alias definitions.
# You may want to put all your additions into a separate file like
# ~/.bash_aliases, instead of adding them here directly.
# See /usr/share/doc/bash-doc/examples in the bash-doc package.

if [ -f ~/.bash_aliases ]; then
. ~/.bash_aliases
fi

# enable programmable completion features (you don't need to enable
# this, if it's already enabled in /etc/bash.bashrc and /etc/profile
# sources /etc/bash.bashrc).
if ! shopt -oq posix; then
if [ -f /usr/share/bash-completion/bash_completion ]; then
. /usr/share/bash-completion/bash_completion
elif [ -f /etc/bash_completion ]; then
. /etc/bash_completion
fi
fi

export PATH="$PATH:/home/farrel/.foundry/bin"
export HADOOP_HOME=~/.hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify
```

3.2.3. Configuration Files

Tahap konfigurasi sistem dilakukan dengan melakukan pengaturan pada empat berkas konfigurasi utama Hadoop yang berada pada direktori:

```
~/hadoop/etc/hadoop/
```

Keempat berkas tersebut berfungsi untuk mendefinisikan filesystem HDFS, struktur direktori penyimpanan data, mode eksekusi MapReduce, serta alokasi sumber daya pada YARN. Konfigurasi ini disesuaikan dengan lingkungan pseudo-distributed yang digunakan, sehingga seluruh komponen Hadoop tetap berjalan dalam proses yang terpisah namun berada pada satu mesin.

a) core-site.xml

Path:

```
nano ~/hadoop/etc/hadoop/core-site.xml
```

Parameter yang dikonfigurasi:

- `fs.defaultFS = hdfs://localhost:9000`
- `hadoop.tmp.dir = /home/farrel/hadoop/tmp`

Konfigurasi `fs.defaultFS` digunakan untuk menetapkan bahwa sistem file utama yang digunakan Hadoop adalah **HDFS** dan NameNode berjalan pada host lokal dengan port 9000. Dengan konfigurasi ini, seluruh perintah yang dijalankan melalui `hdfs dfs` secara otomatis diarahkan ke HDFS, bukan ke filesystem lokal Linux.

Sementara itu, `hadoop.tmp.dir` menentukan direktori penyimpanan file sementara yang digunakan Hadoop selama proses eksekusi. Direktori ditempatkan di dalam folder Hadoop untuk memudahkan pengelolaan, mencegah konflik izin akses, dan memastikan seluruh berkas terkait sistem berada dalam satu lokasi yang terstruktur.

Screenshot:

```
GNU nano 7.2 /home/farrel/hadoop/etc/hadoop/core-site.xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://localhost:9000</value>
  </property>
  <property>
    <name>hadoop.tmp.dir</name>
    <value>/home/farrel/hadoop/tmp</value>
  </property>
</configuration>
```

b) hdfs-site.xml

Path:

```
nano ~/hadoop/etc/hadoop/hdfs-site.xml
```

Parameter yang digunakan:

- `dfs.replication = 1`
- `dfs.namenode.name.dir = /home/farrel/hadoop/data/namenode`
- `dfs.datanode.data.dir = /home/farrel/hadoop/data/datanode`

Nilai `dfs.replication = 1` dipilih karena sistem berjalan dalam mode pseudo-distributed pada satu mesin fisik, sehingga tidak terdapat node cadangan untuk proses replikasi blok data.

Direktori `dfs.namenode.name.dir` digunakan untuk menyimpan metadata HDFS, termasuk namespace dan edit logs. Direktori ini bersifat kritis karena berhubungan langsung dengan struktur sistem file HDFS.

Direktori `dfs.datanode.data.dir` menyimpan blok-blok data yang diproses Hadoop. Pemisahan direktori NameNode dan DataNode dilakukan untuk menjaga keamanan data, memudahkan proses pemeliharaan, serta mengikuti praktik standar konfigurasi Hadoop.

Screenshot:

```
GNU nano 7.2 /home/farrel/hadoop/etc/hadoop/hdfs-site.xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <property>
    <name>dfs.replication</name>
    <value>1</value>
  </property>
  <property>
    <name>dfs.namenode.name.dir</name>
    <value>/home/farrel/hadoop/data/namenode</value>
  </property>
  <property>
    <name>dfs.datanode.data.dir</name>
    <value>/home/farrel/hadoop/data/datanode</value>
  </property>
</configuration>
```

c) mapred-site.xml

Path:

```
nano ~/hadoop/etc/hadoop/mapred-site.xml
```

Parameter yang dikonfigurasi:

- `mapreduce.framework.name = yarn`

Pengaturan ini memastikan bahwa proses MapReduce tidak dijalankan dalam mode lokal, melainkan dikelola oleh **YARN (Yet Another Resource Negotiator)**. Dengan demikian, eksekusi job dilakukan melalui mekanisme penjadwalan yang lebih sesuai dengan model sistem terdistribusi, termasuk dukungan parallel processing dan pemanfaatan resource management.

Screenshot:

```
GNU nano 7.2 /home/farrel/hadoop/etc/hadoop/mapred-site.xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
  <property>
    <name>mapreduce.application.classpath</name>
    <value>$HADOOP_MAPRED_HOME/share/hadoop/mapreduce/*:$HADOOP_MAPRED_>
  </property>
</configuration>
```

d) yarn-site.xml

Path:

```
nano ~/hadoop/etc/hadoop/yarn-site.xml
```

Parameter yang dikonfigurasi:

- `yarn.nodemanager.resource.memory-mb = 8192`
- `yarn.nodemanager.resource.cpu-vcores = 8`

Nilai alokasi memori sebesar **8 GB** dipilih untuk menyeimbangkan kebutuhan Hadoop dengan kapasitas total RAM sistem sebesar 16 GB, sehingga masih tersedia ruang memori bagi proses sistem operasi dan aplikasi lain.

Parameter `cpu-vcores` disetel menjadi **8 logical cores**, dengan tujuan membatasi jumlah thread komputasi yang dijalankan sekaligus serta mencegah kompetisi sumber daya berlebihan antara proses Hadoop dan proses host system di WSL2.

Konfigurasi ini berpengaruh langsung terhadap:

- jumlah mapper dan reducer yang dapat berjalan paralel,
- mekanisme penjadwalan job,
- stabilitas sistem selama proses eksekusi.

Screenshot:

```

GNU nano 7.2 /home/farrel/hadoop/etc/hadoop/yarn-site.xml
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="configuration.xsl"?>
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
  <property>
    <name>yarn.nodemanager.env-whitelist</name>
    <value>JAVA_HOME,HADOOP_COMMON_HOME,HADOOP_HDFS_HOME,HADOOP_CONF_DIR,
  </property>
  <property>
    <name>yarn.nodemanager.resource.memory-mb</name>
    <value>8192</value>
  </property>
  <property>
    <name>yarn.nodemanager.resource.cpu-vcores</name>
    <value>8</value>
  </property>
</configuration>

```

3.2.4. SSH Configuration

Konfigurasi SSH dilakukan untuk memungkinkan Hadoop menjalankan proses internal secara otomatis tanpa memerlukan autentikasi manual (passwordless SSH). Mekanisme ini diperlukan terutama saat Hadoop mengeksekusi service script yang memanggil proses pada node yang sama secara lokal.

Proses konfigurasi dilakukan melalui langkah berikut:

```

sudo apt install openssh-server -y
ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
ssh localhost

```

Penjelasan setiap langkah:

- instalasi **openssh-server** digunakan untuk mengaktifkan layanan SSH di lingkungan WSL2,
- pembuatan keypair RSA dilakukan tanpa passphrase agar dapat digunakan secara otomatis,
- public key ditambahkan ke **authorized_keys** supaya koneksi tidak memerlukan password,
- pengaturan permission file dilakukan untuk mencegah potensi penolakan akses oleh SSH,
- pengujian koneksi dilakukan melalui **ssh localhost**.

Tahapan ini memastikan bahwa Hadoop dapat menjalankan service start-up script secara terintegrasi tanpa interupsi autentikasi.

Screenshot:

```
farrel@LAPTOP-0ROBEU3T:~$ sudo apt install openssh-server -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  libwrap0 ncurses-term openssh-client openssh-sftp-server ssh-import-id
Suggested packages:
  keychain libpam-ssh monkeysphere ssh-askpass molly-guard ufw
The following NEW packages will be installed:
  libwrap0 ncurses-term openssh-server openssh-sftp-server ssh-import-id
The following packages will be upgraded:
  openssh-client
1 upgraded, 5 newly installed, 0 to remove and 110 not upgraded.
Need to get 1786 kB of archives.
After this operation, 6853 kB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 openssh-clie
nt amd64 1:9.6p1-3ubuntu13.14 [906 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 openssh-sftp
-server amd64 1:9.6p1-3ubuntu13.14 [37.3 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble/main amd64 libwrap0 amd64 7.6.g
```

```
farrel@LAPTOP-0ROBEU3T:~$ ssh-keygen -t rsa -P '' -f ~/.ssh/id_rsa
Generating public/private rsa key pair.
Created directory '/home/farrel/.ssh'.
Your identification has been saved in /home/farrel/.ssh/id_rsa
Your public key has been saved in /home/farrel/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:ZTdC/u9p0rg27tJYD50uu2Ir2J28FdtBR80dxdSx+xw farrel@LAPTOP-0ROBEU3T
The key's randomart image is:
+---[RSA 3072]-----+
|      . .o=B      |
|      o  ..o=     |
|      = +  ..     |
|      o = o  .    |
|      S ..o. E    |
|      o+oo .o     |
|      o o =o= . . o|
|      . o O.* +... |
|      oo0**+.o     |
+---[SHA256]-----+
farrel@LAPTOP-0ROBEU3T:~$ cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
farrel@LAPTOP-0ROBEU3T:~$ chmod 600 ~/.ssh/authorized_keys
farrel@LAPTOP-0ROBEU3T:~$ chmod 700 ~/.ssh
farrel@LAPTOP-0ROBEU3T:~$ ssh localhost
The authenticity of host 'localhost (127.0.0.1)' can't be established.
ED25519 key fingerprint is SHA256:+FFoGEN5bMFTX/dBMKeU2gZBXLfUCuSHUipj4Qk7m0s.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 5.15.167.4-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Dec 24 11:22:08 WIB 2025

System load:  0.12               Processes:            32
Usage of /:   0.6% of 1006.85GB  Users logged in:     1
Memory usage: 9%                 IPv4 address for eth0: 172.21.217.131
Swap usage:   0%
```

3.2.5. Format HDFS

Sebelum HDFS digunakan, NameNode perlu melalui proses format untuk membangun struktur awal sistem file Hadoop.

Perintah yang dijalankan:

```
hdfs namenode -format
```

Proses format menghasilkan direktori metadata pada:

```
/home/farrel/hadoop/data/namenode
```

yang ditandai dengan pesan:

“Storage directory ... has been successfully formatted.”

```
2025-12-24 11:22:36,547 INFO common.Storage: Storage directory /home/farrel/hadoop/data/namenode has been successfully formatted.
```

Format NameNode berfungsi untuk:

- membentuk namespace HDFS pertama kali,
- membuat struktur direktori metadata dan edit logs,
- mempersiapkan direktori penyimpanan NameNode sebelum layanan dijalankan.

Tahap ini hanya dilakukan **sekali pada inisialisasi awal**, dan tidak perlu diulang kecuali NameNode di-reset.

Screenshot:

```
farrel@LAPTOP-0ROBEU3T:~$ hdfs namenode -format
WARNING: /home/farrel/hadoop/logs does not exist. Creating.
2025-12-24 11:22:35,533 INFO namenode.NameNode: STARTUP_MSG:
/*****
STARTUP_MSG: Starting NameNode
STARTUP_MSG:   host = LAPTOP-0ROBEU3T/127.0.1.1
STARTUP_MSG:   args = [-format]
STARTUP_MSG:   version = 3.3.6
STARTUP_MSG:   classpath = /home/farrel/hadoop/etc/hadoop:/home/farrel/hadoop/share/hadoop/common/lib/animal-sniffer-annotations-1.17.jar:/home/farrel/hadoop/share/hadoop/common/lib/jetty-webapp-9.4.51.v20230217.jar:/home/farrel/hadoop/share/hadoop/common/lib/netty-transport-classes-kqueue-4.1.89.Final.jar
```

3.2.6. Start Hadoop Services

Setelah konfigurasi selesai dan HDFS diformat, layanan Hadoop dijalankan menggunakan perintah:

```
start-dfs.sh
start-yarn.sh
jps
```

Harapannya disini ketika menjalankan `start-yarn.sh` dan `start-dfs.sh` akan memunculkan NameNode, ResourceManager yang sesuai untuk memastikan instalasi dan berjalan dengan benar.

Screenshot:

```
farrel@LAPTOP-0R0BEU3T:~$ start-yarn.sh
Starting resourcemanager
Starting nodemanagers
farrel@LAPTOP-0R0BEU3T:~$ jps
6962 NodeManager
6163 NameNode
6340 DataNode
6569 SecondaryNameNode
6810 ResourceManager
7403 Jps
```

3.3. Persiapan Dataset pada HDFS

Tahap berikutnya adalah mempersiapkan dataset yang akan diproses oleh program MapReduce dengan mengunggahnya ke sistem penyimpanan HDFS.

3.3.1. Dataset Information

Dataset yang digunakan berasal dari:

Sumber: Kaggle - Web Server Access Logs

URL: <https://www.kaggle.com/datasets/eliasdabbas/web-server-access-logs>

Karakteristik dataset:

- Ukuran: **3.5 GB (3,502,440,823 bytes)**
- Jumlah record: **10,365,152 baris**
- Format: **Apache Common Log Format**

Contoh potongan data:

54.36.149.41 - - [22/Jan/2019:03:56:14 +0330] "GET /filter/" 200 30577

31.56.96.51 - - [22/Jan/2019:03:56:16 +0330] "GET /image/" 200 5667

Alasan pemilihan dataset:

- representatif terhadap traffic log web server,
- ukuran cukup besar untuk menguji performa MapReduce,
- struktur log sesuai proses parsing berbasis pola (pattern extraction).

Dataset ini selanjutnya dipindahkan ke HDFS agar dapat diproses secara terdistribusi.

Screenshot:

```

farrel@LAPTOP-0ROBEU3T:~$ cp "/mnt/c/Users/HP VICTUS/Downloads/access.log" ~/
/access.log
farrel@LAPTOP-0ROBEU3T:~$ ls -lh ~/access.log
-rwxr-xr-x 1 farrel farrel 3.3G Dec 24 11:34 /home/farrel/access.log
farrel@LAPTOP-0ROBEU3T:~$ head -5 ~/access.log
54.36.149.41 - - [22/Jan/2019:03:56:14 +0330] "GET /filter/27|13%20%D9%85%DA
%AF%D8%A7%D9%BE%DB%8C%DA%A9%D8%B3%D9%84,27|%DA%A9%D9%85%D8%AA%D8%B1%20%D8%A7
%D8%B2%205%20%D9%85%DA%AF%D8%A7%D9%BE%DB%8C%DA%A9%D8%B3%D9%84,p53 HTTP/1.1"
200 30577 "-" "Mozilla/5.0 (compatible; AhrefsBot/6.1; +http://ahrefs.com/ro
bot/)" "-"
31.56.96.51 - - [22/Jan/2019:03:56:16 +0330] "GET /image/60844/productModel/
200x200 HTTP/1.1" 200 5667 "https://www.zanbil.ir/m/filter/b113" "Mozilla/5.
0 (Linux; Android 6.0; ALE-L21 Build/HuaweiALE-L21) AppleWebKit/537.36 (KHTML
L, like Gecko) Chrome/66.0.3359.158 Mobile Safari/537.36" "-"
31.56.96.51 - - [22/Jan/2019:03:56:16 +0330] "GET /image/61474/productModel/
200x200 HTTP/1.1" 200 5379 "https://www.zanbil.ir/m/filter/b113" "Mozilla/5.
0 (Linux; Android 6.0; ALE-L21 Build/HuaweiALE-L21) AppleWebKit/537.36 (KHTML
L, like Gecko) Chrome/66.0.3359.158 Mobile Safari/537.36" "-"
40.77.167.129 - - [22/Jan/2019:03:56:17 +0330] "GET /image/14925/productMode
l/100x100 HTTP/1.1" 200 1696 "-" "Mozilla/5.0 (compatible; bingbot/2.0; +htt
p://www.bing.com/bingbot.htm)" "-"
91.99.72.15 - - [22/Jan/2019:03:56:17 +0330] "GET /product/31893/62100/%D8%B
3%D8%B4%D9%88%D8%A7%D8%B1-%D8%AE%D8%A7%D9%86%DA%AF%DB%8C-%D9%BE%D8%B1%D9%86%
D8%B3%D9%84%DB%8C-%D9%85%D8%AF%D9%84-PR257AT HTTP/1.1" 200 41483 "-" "Mozill
a/5.0 (Windows NT 6.2; Win64; x64; rv:16.0) Gecko/16.0 Firefox/16.0" "-"
farrel@LAPTOP-0ROBEU3T:~$ wc -l ~/access.log
10365152 /home/farrel/access.log

```

3.3.2. Upload to HDFS

Proses unggah dataset dilakukan melalui tahapan berikut:

1) Membuat direktori HDFS

```
hdfs dfs -mkdir -p /user/farrel/input
```

Direktori ini berfungsi sebagai lokasi penyimpanan dataset input.

2) Mengunggah dataset ke HDFS

```
hdfs dfs -put ~/access.log /user/farrel/input/
```

dataset dipindahkan dari files sistem lokal ke HDFS.

3) Verifikasi file

```
hdfs dfs -ls /user/farrel/input/
hdfs dfs -du -h /user/farrel/input/
```

Screenshot:

```

farrel@LAPTOP-0ROBEU3T:~$ wc -l ~/access.log
10365152 /home/farrel/access.log
farrel@LAPTOP-0ROBEU3T:~$ hdfs dfs -mkdir -p /user/farrel/input
2025-12-24 15:18:02,100 WARN util.NativeCodeLoader: Unable to load native-ha
doop library for your platform... using builtin-java classes where applicabl
e
farrel@LAPTOP-0ROBEU3T:~$ hdfs dfs -put ~/access.log /user/farrel/input/
2025-12-24 15:19:24,409 WARN util.NativeCodeLoader: Unable to load native-ha
doop library for your platform... using builtin-java classes where applicabl
e
farrel@LAPTOP-0ROBEU3T:~$ hdfs dfs -ls /user/farrel/input/
2025-12-24 15:20:37,902 WARN util.NativeCodeLoader: Unable to load native-ha
doop library for your platform... using builtin-java classes where applicabl
e
Found 1 items
-rw-r--r--  1 farrel supergroup 3502440823 2025-12-24 15:20 /user/farrel/in
put/access.log

```

Keluaran verifikasi menunjukkan:

```
-rw-r--r-- 1 farrel supergroup 3.5G /user/farrel/input/access.log
```

Hal ini menandakan bahwa:

- dataset berhasil diunggah,
- ukuran file sesuai dengan data asli,
- file telah siap digunakan sebagai input job MapReduce.

3.4. Implementasi Program MapReduce

3.4.1. Project Structure

Implementasi program MapReduce dilakukan menggunakan bahasa pemrograman Java dengan struktur proyek sebagai berikut:

```

~/mapreduce-project/
├── src/
│   ├── IPCountMapper.java
│   ├── IPCountReducer.java
│   └── LogAnalysisDriver.java
├── output/
└── loganalysis.jar

```

Struktur ini mengikuti pola standar aplikasi Hadoop, di mana komponen Mapper, Reducer, dan Driver dipisahkan agar mudah dikelola serta memungkinkan proses kompilasi dan packaging dilakukan secara modular.

3.4.2. Mapper Implementation

```
src/IPCountMapper.java
```

Fungsi:

- membaca setiap baris log,
- mengekstrak alamat IP (token pertama sebelum spasi),
- menghasilkan pasangan output berupa: <IP_Address, 1>

Mapper menerima input dalam bentuk:

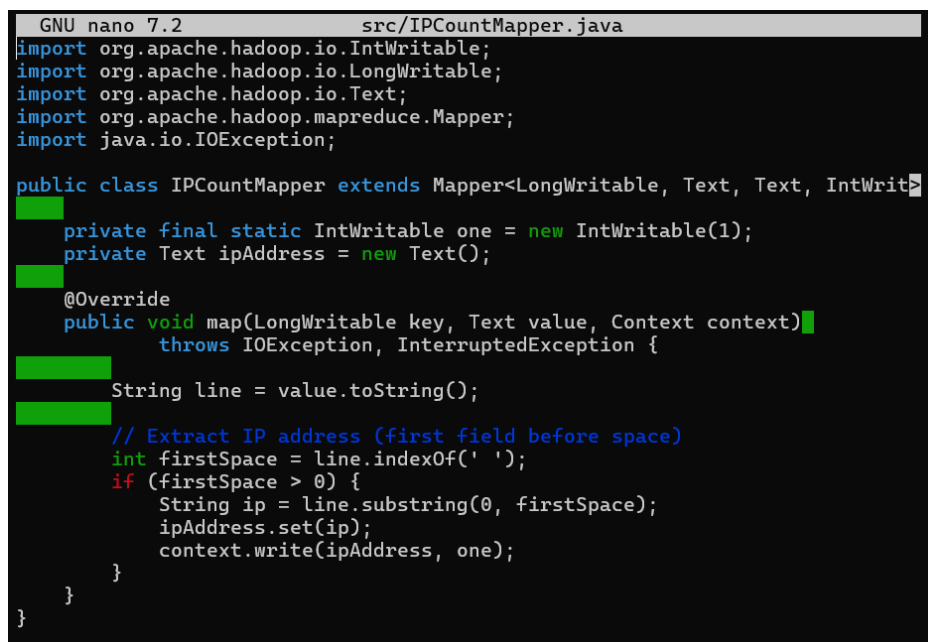
- <LongWritable, Text> — nomor baris dan isi baris log

dan menghasilkan output:

- <Text, IntWritable> — IP Address dan nilai hitungan awal.

Log parsing dilakukan dengan memotong string sampai karakter spasi pertama, sehingga hanya alamat IP yang diproses sebagai key. Pendekatan ini sesuai dengan karakteristik **Apache Common Log Format** yang digunakan dataset.

Screenshot:



```
GNU nano 7.2      src/IPCountMapper.java
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
import java.io.IOException;

public class IPCountMapper extends Mapper<LongWritable, Text, Text, IntWritable> {
    private final static IntWritable one = new IntWritable(1);
    private Text ipAddress = new Text();

    @Override
    public void map(LongWritable key, Text value, Context context)
        throws IOException, InterruptedException {
        String line = value.toString();

        // Extract IP address (first field before space)
        int firstSpace = line.indexOf(' ');
        if (firstSpace > 0) {
            String ip = line.substring(0, firstSpace);
            ipAddress.set(ip);
            context.write(ipAddress, one);
        }
    }
}
```

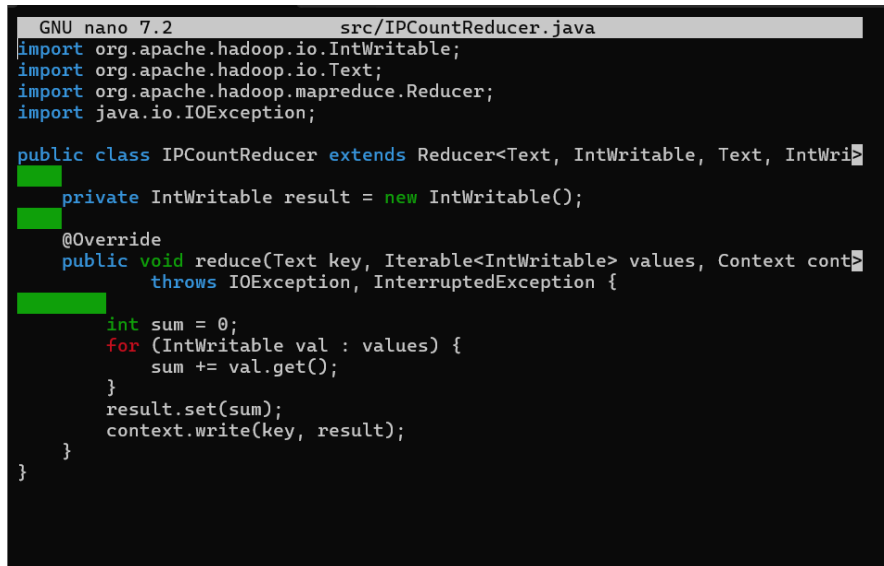
3.4.3. Reducer Implementation

src/IPCountReducer.java

Fungsi:

- menerima sekumpulan nilai count untuk setiap IP
- menjumlahkan seluruh nilai
- Output: <IP_Address, Total_Count>

Screenshot:



```
GNU nano 7.2 src/IPCountReducer.java
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
import java.io.IOException;

public class IPCountReducer extends Reducer<Text, IntWritable, Text, IntWritable> {
    private IntWritable result = new IntWritable();

    @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
        int sum = 0;
        for (IntWritable val : values) {
            sum += val.get();
        }
        result.set(sum);
        context.write(key, result);
    }
}
```

Input tipe data:

<Text, Iterable<IntWritable>>

Output tipe data:

<Text, IntWritable>

Proses agregasi dilakukan melalui perulangan sederhana (loop) untuk menambahkan seluruh nilai count. Hasil akhir berupa total frekuensi kemunculan setiap alamat IP dalam dataset.

Reducer berfungsi sebagai tahap finalisasi perhitungan dan merepresentasikan proses **grouping + aggregation** dalam arsitektur MapReduce.

3.4.4. Driver Implementation

src/LogAnalysisDriver.java

Driver berperan sebagai:

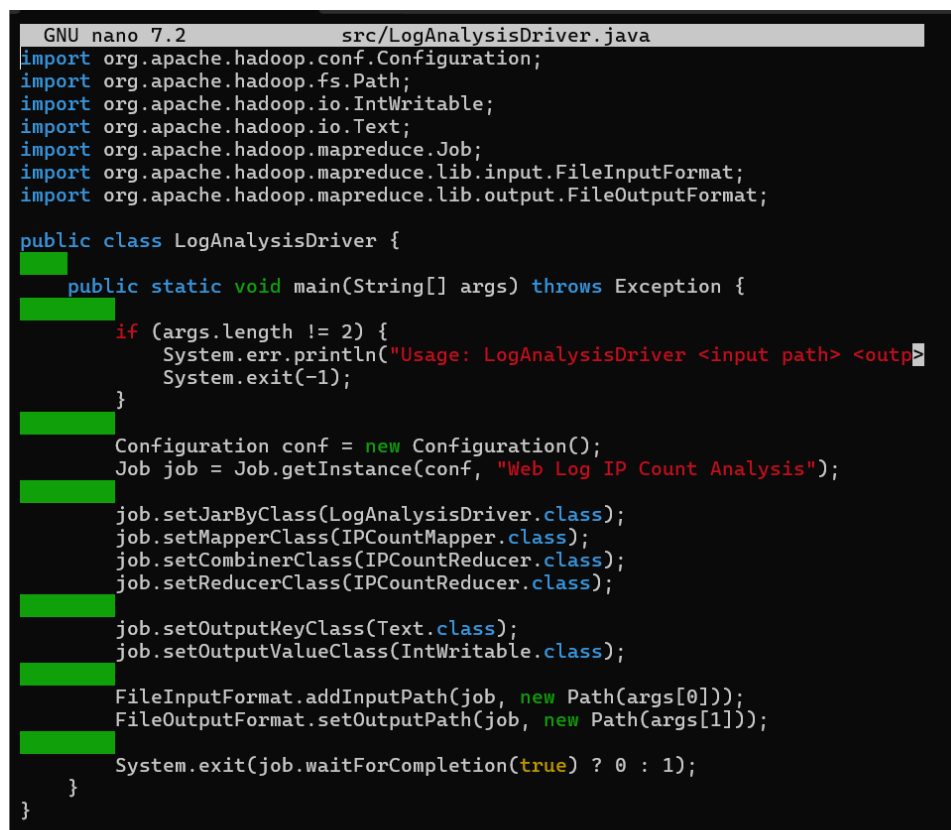
- pengatur konfigurasi job MapReduce,
- penghubung antara input, proses, dan output,
- inisiasi eksekusi job melalui Hadoop runtime.

Fungsi utama Driver:

- menentukan kelas Mapper, Combiner, dan Reducer,
- mendefinisikan format input dan output,
- menetapkan lokasi input HDFS dan direktori output,
- mengeksekusi job hingga selesai.

Program juga menggunakan **Reducer sebagai Combiner** untuk melakukan agregasi awal di sisi Mapper, sehingga jumlah data yang dikirim ke tahap Reduce menjadi lebih kecil. Hal ini memberikan efisiensi pada proses shuffle dan transfer data antar node virtual.

Screenshot:



```
GNU nano 7.2 src/LogAnalysisDriver.java
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class LogAnalysisDriver {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: LogAnalysisDriver <input path> <output path>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Web Log IP Count Analysis");

        job.setJarByClass(LogAnalysisDriver.class);
        job.setMapperClass(IPCountMapper.class);
        job.setCombinerClass(IPCountReducer.class);
        job.setReducerClass(IPCountReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

3.5. Kompilasi dan Packaging

3.5.1. Kompilasi

```
javac -classpath $(hadoop classpath) -d output src/*.java
```

Penjelasan:

- classpath Hadoop digunakan agar seluruh dependency dikenali saat kompilasi,

- opsi `-d output` mengarahkan hasil kompilasi ke folder `output`,
- seluruh berkas `.java` di dalam direktori `src` dikompilasi secara bersama.

Hasil kompilasi berupa berkas `.class`, antara lain:

- `IPCountMapper.class`
- `IPCountReducer.class`
- `LogAnalysisDriver.class`

Screenshot:

```
farrel@LAPTOP-0ROBEU3T:~/mapreduce-project$ mkdir -p output
farrel@LAPTOP-0ROBEU3T:~/mapreduce-project$ javac -classpath $(hadoop classp
ath) -d output src/*.java
farrel@LAPTOP-0ROBEU3T:~/mapreduce-project$ ls -l output/
total 12
-rw-r--r-- 1 farrel farrel 1811 Dec 24 15:29 IPCountMapper.class
-rw-r--r-- 1 farrel farrel 1693 Dec 24 15:29 IPCountReducer.class
-rw-r--r-- 1 farrel farrel 1595 Dec 24 15:29 LogAnalysisDriver.class
farrel@LAPTOP-0ROBEU3T:~/mapreduce-project$ cd output
farrel@LAPTOP-0ROBEU3T:~/mapreduce-project/output$ jar -cvf loganalysis.jar
*.class
added manifest
adding: IPCountMapper.class(in = 1811) (out= 748)(deflated 58%)
adding: IPCountReducer.class(in = 1693) (out= 712)(deflated 57%)
adding: LogAnalysisDriver.class(in = 1595) (out= 885)(deflated 44%)
```

3.5.2. JAR Packaging

Langkah berikutnya adalah melakukan packaging menjadi file JAR:

```
cd output
jar -cvf loganalysis.jar *.class
cd ..
mv output/loganalysis.jar .
```

Tujuan packaging:

- menghasilkan aplikasi MapReduce dalam satu unit executable,
- memungkinkan job dipanggil melalui perintah `hadoop jar`,
- mempermudah deployment dan pengelolaan program.

File akhir yang dihasilkan: `loganalysis.jar` (3.1 KB)

Screenshot:

```

farrel@LAPTOP-0ROBEU3T:~/mapreduce-project$ cd output
farrel@LAPTOP-0ROBEU3T:~/mapreduce-project/output$ jar -cvf loganalysis.jar
*.class
added manifest
adding: IPCountMapper.class(in = 1811) (out= 748)(deflated 58%)
adding: IPCountReducer.class(in = 1693) (out= 712)(deflated 57%)
adding: LogAnalysisDriver.class(in = 1595) (out= 885)(deflated 44%)
farrel@LAPTOP-0ROBEU3T:~/mapreduce-project/output$ cd ..
farrel@LAPTOP-0ROBEU3T:~/mapreduce-project$ mv output/loganalysis.jar .
farrel@LAPTOP-0ROBEU3T:~/mapreduce-project$ ls -lh loganalysis.jar
-rw-r--r-- 1 farrel farrel 3.1K Dec 24 15:29 loganalysis.jar
farrel@LAPTOP-0ROBEU3T:~/mapreduce-project$ hdfs dfs -rm -r /user/farrel/out
put
2025-12-24 15:30:42,128 WARN util.NativeCodeLoader: Unable to load native-ha
doop library for your platform... using builtin-java classes where applicabl
e
rm: `/user/farrel/output': No such file or directory

```

3.6. Eksekusi Job MapReduce

3.6.1. Submit Job

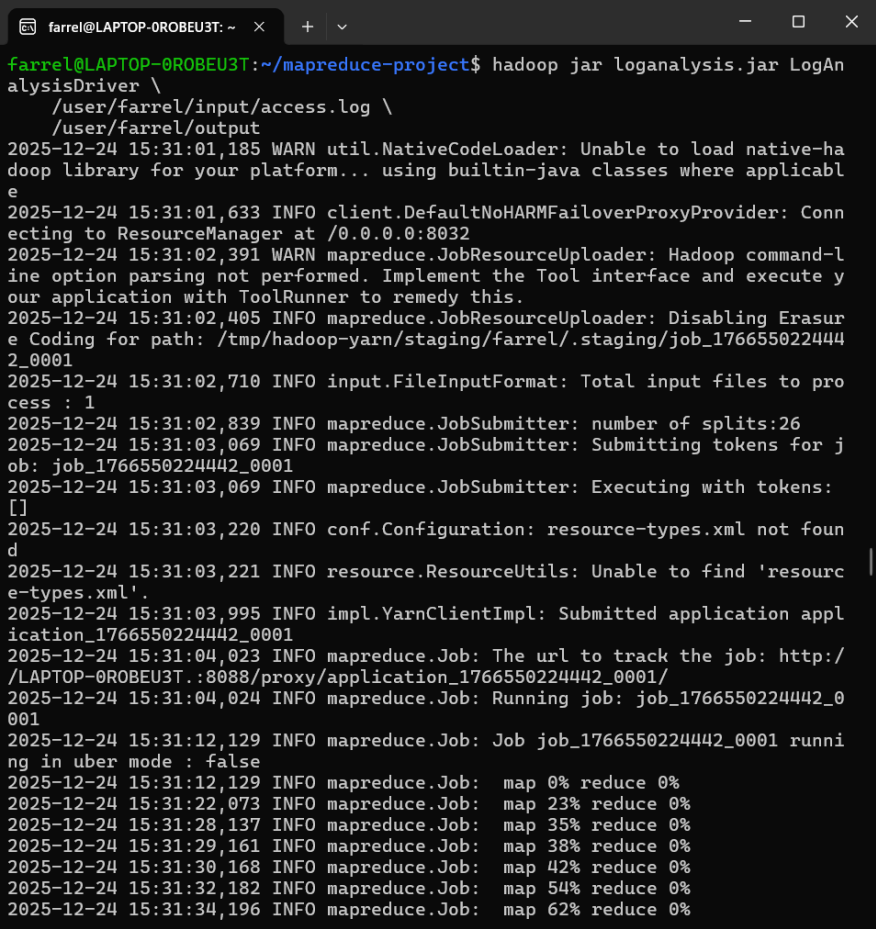
Job dieksekusi di lingkungan Hadoop dengan menggunakan perintah:

```

hadoop jar loganalysis.jar LogAnalysisDriver \
/user/farrel/input/access.log \
/user/farrel/output

```

Screenshot:



```

farrel@LAPTOP-0ROBEU3T:~/mapreduce-project$ hadoop jar loganalysis.jar LogAn
alysisDriver \
/user/farrel/input/access.log \
/user/farrel/output
2025-12-24 15:31:01,185 WARN util.NativeCodeLoader: Unable to load native-ha
doop library for your platform... using builtin-java classes where applicabl
e
2025-12-24 15:31:01,633 INFO client.DefaultNoHARMFailoverProxyProvider: Conn
ecting to ResourceManager at /0.0.0.0:8032
2025-12-24 15:31:02,391 WARN mapreduce.JobResourceUploader: Hadoop command-l
ine option parsing not performed. Implement the Tool interface and execute y
our application with ToolRunner to remedy this.
2025-12-24 15:31:02,405 INFO mapreduce.JobResourceUploader: Disabling Erasu
re Coding for path: /tmp/hadoop-yarn/staging/farrel/.staging/job_176655022444
2_0001
2025-12-24 15:31:02,710 INFO input.FileInputFormat: Total input files to pro
cess : 1
2025-12-24 15:31:02,839 INFO mapreduce.JobSubmitter: number of splits:26
2025-12-24 15:31:03,069 INFO mapreduce.JobSubmitter: Submitting tokens for j
ob: job_1766550224442_0001
2025-12-24 15:31:03,069 INFO mapreduce.JobSubmitter: Executing with tokens:
[]
2025-12-24 15:31:03,220 INFO conf.Configuration: resource-types.xml not foun
d
2025-12-24 15:31:03,221 INFO resource.ResourceUtils: Unable to find 'resourc
e-types.xml'.
2025-12-24 15:31:03,995 INFO impl.YarnClientImpl: Submitted application appl
ication_1766550224442_0001
2025-12-24 15:31:04,023 INFO mapreduce.Job: The url to track the job: http:/
/LAPTOP-0ROBEU3T:8088/proxy/application_1766550224442_0001/
2025-12-24 15:31:04,024 INFO mapreduce.Job: Running job: job_1766550224442_0
001
2025-12-24 15:31:12,129 INFO mapreduce.Job: Job job_1766550224442_0001 runni
ng in uber mode : false
2025-12-24 15:31:12,129 INFO mapreduce.Job: map 0% reduce 0%
2025-12-24 15:31:22,073 INFO mapreduce.Job: map 23% reduce 0%
2025-12-24 15:31:28,137 INFO mapreduce.Job: map 35% reduce 0%
2025-12-24 15:31:29,161 INFO mapreduce.Job: map 38% reduce 0%
2025-12-24 15:31:30,168 INFO mapreduce.Job: map 42% reduce 0%
2025-12-24 15:31:32,182 INFO mapreduce.Job: map 54% reduce 0%
2025-12-24 15:31:34,196 INFO mapreduce.Job: map 62% reduce 0%

```

Parameter pertama:

- input dataset pada HDFS

Parameter kedua:

- direktori output yang akan dibuat Hadoop.

Selama proses eksekusi, Hadoop akan:

- membagi dataset menjadi sejumlah blok,
- menjalankan Mapper secara paralel,
- melakukan shuffle & sort hasil mapper,
- menjalankan Reducer untuk agregasi final.

3.6.2. Job Progress

Progress Log:

15:31:12 map 0% reduce 0%

15:31:22 map 23% reduce 0%

15:31:32 map 54% reduce 0%

15:31:38 map 81% reduce 0%

15:31:39 map 81% reduce 24%

15:31:43 map 100% reduce 24%

15:31:44 map 100% reduce 100%

15:31:45 Job completed successfully

Progress log memperlihatkan perkembangan job pada setiap interval, mencerminkan kinerja parallel processing yang dilakukan oleh Hadoop.

Total waktu penyelesaian: 44 seconds

Screenshot:

```
farrel@LAPTOP-0ROBEU3T: ~  
2025-12-24 15:31:04,024 INFO mapreduce.Job: Running job: job_1766550224442_0001  
2025-12-24 15:31:12,129 INFO mapreduce.Job: Job job_1766550224442_0001 running in uber mode : false  
2025-12-24 15:31:12,129 INFO mapreduce.Job: map 0% reduce 0%  
2025-12-24 15:31:22,073 INFO mapreduce.Job: map 23% reduce 0%  
2025-12-24 15:31:28,137 INFO mapreduce.Job: map 35% reduce 0%  
2025-12-24 15:31:29,161 INFO mapreduce.Job: map 38% reduce 0%  
2025-12-24 15:31:30,168 INFO mapreduce.Job: map 42% reduce 0%  
2025-12-24 15:31:32,182 INFO mapreduce.Job: map 54% reduce 0%  
2025-12-24 15:31:34,196 INFO mapreduce.Job: map 62% reduce 0%  
2025-12-24 15:31:36,211 INFO mapreduce.Job: map 69% reduce 0%  
2025-12-24 15:31:37,218 INFO mapreduce.Job: map 73% reduce 0%  
2025-12-24 15:31:38,227 INFO mapreduce.Job: map 81% reduce 0%  
2025-12-24 15:31:39,233 INFO mapreduce.Job: map 81% reduce 24%  
2025-12-24 15:31:40,241 INFO mapreduce.Job: map 88% reduce 24%  
2025-12-24 15:31:41,252 INFO mapreduce.Job: map 92% reduce 24%  
2025-12-24 15:31:43,263 INFO mapreduce.Job: map 100% reduce 24%  
2025-12-24 15:31:44,272 INFO mapreduce.Job: map 100% reduce 100%  
2025-12-24 15:31:45,284 INFO mapreduce.Job: Job job_1766550224442_0001 completed successfully  
2025-12-24 15:31:45,475 INFO mapreduce.Job: Counters: 55  
  File System Counters  
    FILE: Number of bytes read=6162404  
    FILE: Number of bytes written=19770790  
    FILE: Number of read operations=0  
    FILE: Number of large read operations=0  
    FILE: Number of write operations=0  
    HDFS: Number of bytes read=3502546213  
    HDFS: Number of bytes written=4159283  
    HDFS: Number of read operations=83  
    HDFS: Number of large read operations=0  
    HDFS: Number of write operations=2  
    HDFS: Number of bytes read erasure-coded=0  
  Job Counters  
    Killed map tasks=1  
    Launched map tasks=26  
    Launched reduce tasks=1  
    Data-local map tasks=26  
    Total time spent by all maps in occupied slots (ms)=117673  
    Total time spent by all reduces in occupied slots (ms)=19612
```

3.6.3. Job Counters

Beberapa metrik penting yang dihasilkan:

- Map input records: 10,365,152
- Map output records: 10,365,152
- Combine input records: 10,365,152
- Combine output records: 314,136
- Reduce input records: 314,136
- Reduce output records: 258,606

Combiner Efficiency: 97.0% data reduction

Nilai tersebut menunjukkan bahwa:

- setiap baris log diproses sebagai satu record Map,
- Combiner berhasil menurunkan jumlah record secara signifikan,

- jumlah IP unik yang terbentuk setelah Reduce adalah **258.606** entri.

Screenshot:

```

Map-Reduce Framework
  Map input records=10365152
  Map output records=10365152
  Map output bytes=183258630
  Map output materialized bytes=6162554
  Input split bytes=2990
  Combine input records=10365152
  Combine output records=314136
  Reduce input groups=258606
  Reduce shuffle bytes=6162554
  Reduce input records=314136
  Reduce output records=258606
  Spilled Records=628272
  Shuffled Maps =26
  Failed Shuffles=0
  Merged Map outputs=26
  GC time elapsed (ms)=2260
  CPU time spent (ms)=114440
  Physical memory (bytes) snapshot=9967931392
  Virtual memory (bytes) snapshot=74984718336
  Total committed heap usage (bytes)=6748635136
  Peak Map Physical memory (bytes)=533430272
  Peak Map Virtual memory (bytes)=2784747520
  Peak Reduce Physical memory (bytes)=285995008
  Peak Reduce Virtual memory (bytes)=2772385792
Shuffle Errors
  BAD_ID=0
  CONNECTION=0
  IO_ERROR=0
  WRONG_LENGTH=0
  WRONG_MAP=0
  WRONG_REDUCE=0
File Input Format Counters
  Bytes Read=3502543223
File Output Format Counters
  Bytes Written=4159283

```

3.7. Monitoring dan Verifikasi

Setelah job MapReduce berhasil dijalankan, tahap berikutnya adalah melakukan proses monitoring dan verifikasi hasil. Monitoring dilakukan menggunakan antarmuka YARN Web UI, sedangkan verifikasi hasil dilakukan melalui pengecekan file output pada HDFS serta analisis konten keluaran job.

Tahap ini bertujuan untuk memastikan bahwa:

- job dieksekusi tanpa error,
- seluruh proses Map dan Reduce berjalan hingga selesai,
- hasil output sesuai dengan struktur dan tujuan analisis,
- tidak terdapat kegagalan proses pada sisi resource management.

3.7.1. YARN Web UI

Proses monitoring dilakukan melalui halaman: <http://localhost:8088>

Screenshot:



Cluster

About
Nodes
Node Labels
Applications
NEW
NEW SAVING
SUBMITTED
ACCEPTED
RUNNING
FINISHED
FAILED
KILLED
Scheduler

Tools

Cluster Metrics

Apps Submitted	0	Apps Pending	1	Apps Running	0	Apps Completed	1	Containers Running	Used Resources			
1		0		1		0		1		<memory:2 GB, vCores:1>		<memory:8 GB, vCores:4>

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes		Decommissioned Nodes		Lost Nodes
1	0		0		0

Scheduler Metrics

Scheduler Type		Scheduling Resource Type		Minimum Allocation		Maximum Allocation	
Capacity Scheduler		[memory-mb (unit=Mi), vcores]		<memory:1024, vCores:1>		<memory:8192, vCores:4>	

Show 20 entries

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Running Containers	FinalContainers
application_1766550224442_0001	farrel	Web Log IP Count Analysis	MAPREDUCE		default	0	Wed Dec 24 15:31:03 +0700 2025	Wed Dec 24 15:31:06 +0700 2025	N/A	RUNNING	UNDEFINED	1	1

Showing 1 to 1 of 1 entries

Cluster Metrics

Apps Submitted	0	Apps Pending	1	Apps Running	0	Apps Completed	1	Containers Running	Used Resources			
1		0		1		0		1		<memory:2 GB, vCores:1>		<memory:8 GB, vCores:4>

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes		Decommissioned Nodes		Lost Nodes
1	0		0		0

Scheduler Metrics

Scheduler Type		Scheduling Resource Type		Minimum Allocation		Maximum Allocation	
Capacity Scheduler		[memory-mb (unit=Mi), vcores]		<memory:1024, vCores:1>		<memory:8192, vCores:4>	

Show 20 entries

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Running Containers	FinalContainers
application_1766550224442_0001	farrel	Web Log IP Count Analysis	MAPREDUCE		default	0	Wed Dec 24 15:31:03 +0700 2025	Wed Dec 24 15:31:06 +0700 2025	N/A	RUNNING	UNDEFINED	1	1

Showing 1 to 1 of 1 entries

Apps Running	Used Resources		Total Resources		Reserved Resources		Physical Mem Used %		Physical VCores Used %	
<memory:2 GB, vCores:1>		<memory:8 GB, vCores:8>		<memory:0 B, vCores:0>		41		0		

Decommissioned Nodes	Lost Nodes	Unhealthy Nodes	Rebooted Nodes	Shutdown Nodes
0	0	0	0	0

Minimum Allocation		Maximum Allocation		Maximum Cluster Application Priority		Scheduler Busy %	
24, vCores:1>		<memory:8192, vCores:4>		0		0	

Show 20 entries

LaunchTime	FinishTime	State	FinalStatus	Running Containers	Allocated CPU VCores	Allocated Memory MB	Allocated GPUs	Reserved CPU VCores	Reserved Memory MB	Reserved GPUs	% of Queue	% of Cluster	Progress	Tracking UI	Blacklisted Nodes
Wed Dec 24 15:31:06 +0700 2025	N/A	RUNNING	UNDEFINED	1	1	2048	-1	0	0	-1	25.0	25.0		ApplicationMaster	0

First Previous 1 Next Last

Antarmuka YARN Web UI menampilkan informasi eksekusi job, antara lain:

- Application ID
- Status eksekusi (FINISHED / SUCCEDED)
- Progress eksekusi (100%)
- Total waktu eksekusi job

Berdasarkan tampilan monitoring, job tercatat berada pada kondisi:

- **State:** FINISHED
- **Final Status:** SUCCEDED
- **Progress:** 100%
- **Elapsed Time:** ± 44 detik

Hal ini menunjukkan bahwa proses MapReduce berjalan tanpa kegagalan pada tahap map, shuffle, hingga reduce, serta seluruh resource container berhasil dialokasikan dan dilepaskan kembali secara normal setelah job selesai.

3.7.2. Output Verification

Verifikasi hasil dilakukan melalui HDFS dengan langkah berikut:

1) Melihat daftar file output

```
hdfs dfs -ls /user/farrel/output/
```

2) Melihat sebagian isi hasil output

```
hdfs dfs -cat /user/farrel/output/part-r-00000 | head -20
```

3) Mengunduh hasil ke filesystem lokal

```
hdfs dfs -get /user/farrel/output/part-r-00000 ~/mapreduce_result.txt
```

Direktori output menghasilkan dua berkas:

- **_SUCCESS**
berfungsi sebagai indikator bahwa job selesai tanpa error,
- **part-r-00000**
berisi hasil akhir proses Reduce.

File **_SUCCESS** berukuran 0 byte dan tidak menyimpan data, namun berfungsi sebagai marker status eksekusi yang dihasilkan otomatis oleh Hadoop.

Sementara itu file **part-r-00000** berisi pasangan:

```
<IP_Address, Total_Request_Count>
```

dengan ukuran sekitar **4.1 MB**, sesuai dengan jumlah IP unik yang berhasil dihasilkan.

Screenshot:

```

farrel@LAPTOP-0R0BEU3T:~/mapreduce-project$ hdfs dfs -ls /user/farrel/output
/
2025-12-24 15:42:00,172 WARN util.NativeCodeLoader: Unable to load native-ha
doop library for your platform... using builtin-java classes where applicabl
e
Found 2 items
-rw-r--r-- 1 farrel supergroup 0 2025-12-24 15:31 /user/farrel/ou
tput/_SUCCESS
-rw-r--r-- 1 farrel supergroup 4159283 2025-12-24 15:31 /user/farrel/ou
tput/part-r-00000
farrel@LAPTOP-0R0BEU3T:~/mapreduce-project$ hdfs dfs -cat /user/farrel/oupu
t/part-r-00000 | head -20
2025-12-24 15:42:10,748 WARN util.NativeCodeLoader: Unable to load native-ha
doop library for your platform... using builtin-java classes where applicabl
e
1.132.107.223 15
1.132.108.133 1
1.136.111.52 1
1.158.74.178 37
1.159.185.202 1
1.214.221.2 2
1.215.205.157 2
1.234.99.77 108
1.36.128.224 34
1.43.175.52 1
1.46.105.255 11
1.46.167.44 11
1.46.76.228 22
1.46.9.141 11
1.52.109.77 1
10.1.48.115 125
10.1.52.71 147
10.1.68.25 446
10.10.56.92 282
10.103.61.149 56
cat: Unable to write to output stream.

```

3.7.3. Sample Results

Cuplikan hasil output menunjukkan beberapa alamat IP dengan frekuensi request tertinggi, misalnya:

```

66.249.66.194 353483
66.249.66.91 314522
151.239.241.163 92475
66.249.66.92 88332

```

Nilai tersebut menggambarkan:

- adanya IP tertentu yang mendominasi jumlah request,
- kemungkinan trafik berasal dari crawler, bot, atau automated client,
- distribusi request bersifat tidak merata (heavy-tailed behavior).

Informasi ini dapat digunakan sebagai dasar analisis lebih lanjut, seperti:

- deteksi aktivitas intensif,
- identifikasi aktivitas bot traffic,
- analisis beban akses berdasarkan kelompok IP.

Screenshot:

```
farrel@LAPTOP-0ROBEU3T:~/mapreduce-project$ hdfs dfs -cat /user/farrel/output/
t/part-r-000000 | sort -t$'\t' -k2 -nr | head -10
2025-12-24 15:42:17,199 WARN util.NativeCodeLoader: Unable to load native-ha
doop library for your platform... using builtin-java classes where applicabl
e
66.249.66.194      353483
66.249.66.91      314522
151.239.241.163   92475
66.249.66.92      88332
91.99.30.32       45979
104.222.32.91     42058
91.99.72.15       38694
91.99.47.57       38612
5.78.190.233      37204
195.181.168.181   27979
```

3.7.4. Total Unique IPs

Jumlah total IP unik dihitung menggunakan perintah:

```
hdfs dfs -cat /user/farrel/output/part-r-000000 | wc -l
```

Output menunjukkan:

- Total unique IP addresses: **258.606**

Interpretasi hasil:

- total request yang diproses: **10.365.152 baris log**
- jumlah IP unik: **258.606 entitas**
- rata-rata request per IP $\approx$ **40 permintaan per IP**

Nilai ini menunjukkan bahwa:

- sebagian besar IP hanya melakukan akses dalam jumlah kecil,
- namun terdapat beberapa IP dengan volume request yang jauh lebih tinggi,
- fenomena ini konsisten dengan karakteristik **distribusi akses log web**, di mana sebagian kecil client menghasilkan sebagian besar trafik.

Kesimpulan Tahap Monitoring & Verifikasi

Berdasarkan hasil monitoring dan validasi output:

- job MapReduce berhasil dieksekusi dengan status **SUCCEEDED**,

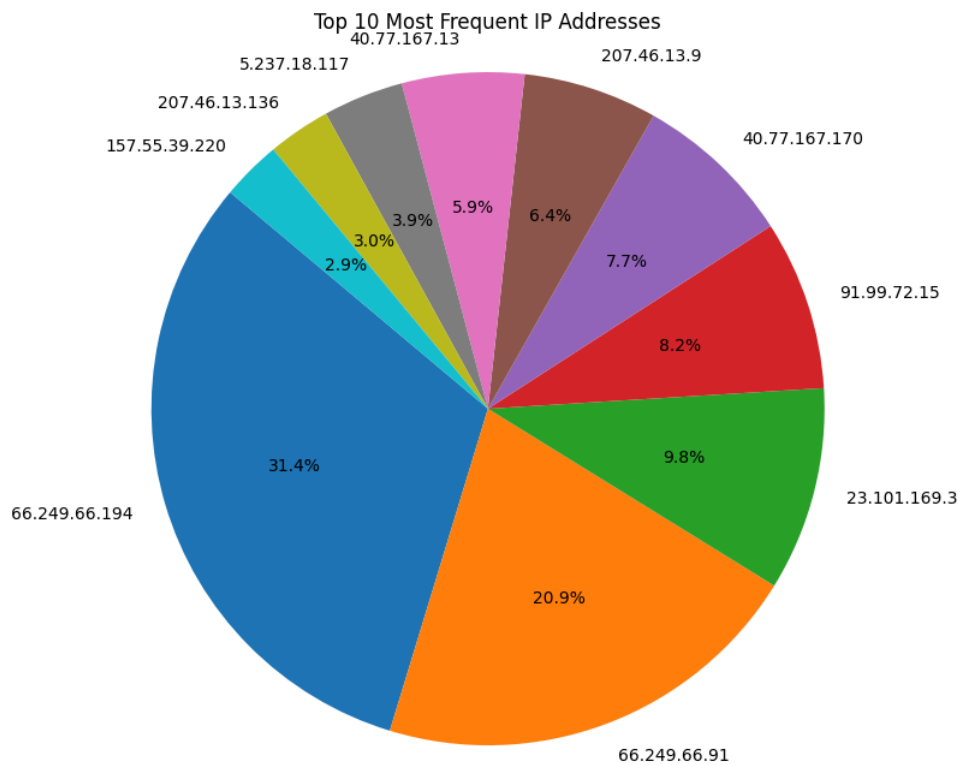
- seluruh komponen layanan Hadoop berjalan stabil,
- hasil output tervalidasi melalui YARN UI dan verifikasi HDFS,
- struktur data sesuai dengan tujuan analisis frekuensi IP.

BAB IV

4. IMPLEMENTASI SISTEM

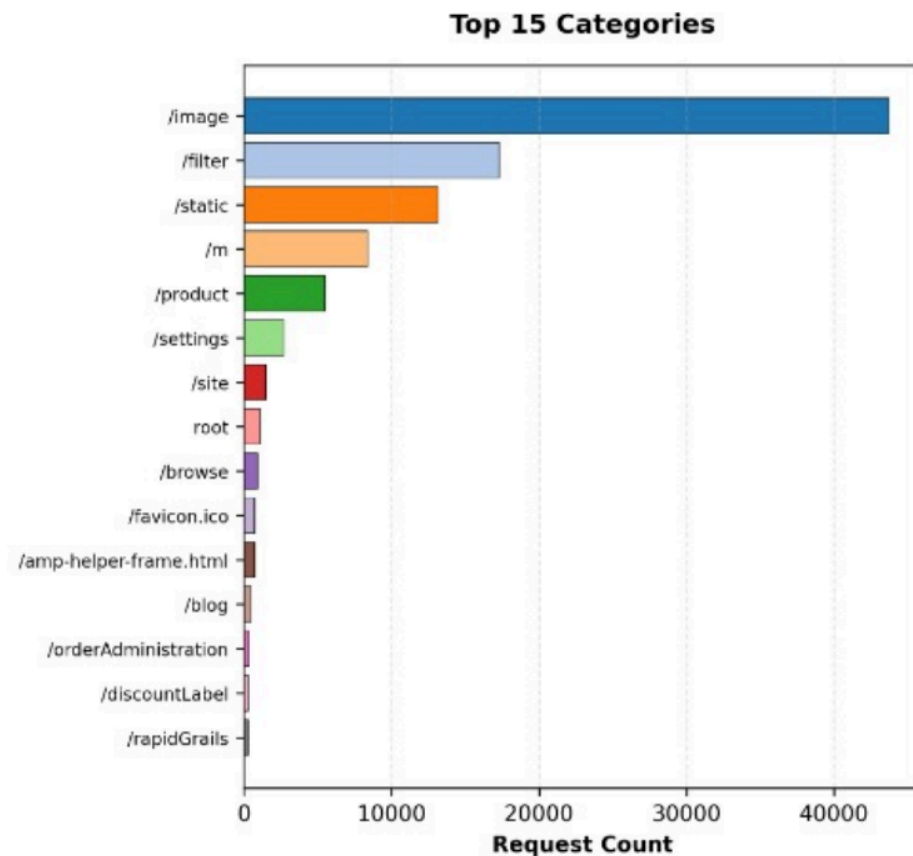
4.1. Pembahasan

Berdasarkan hasil reduksi data yang telah dilakukan sebelumnya, dapat disimpulkan bahwa data tersebut menunjukkan pola aktivitas yang cukup jelas. Mayoritas permintaan (request) berasal dari berbagai alamat IP yang berbeda, namun setelah dianalisis lebih lanjut, terlihat bahwa sebagian besar trafik didominasi oleh beberapa IP tertentu. Dua di antaranya adalah IP 66.249.66.194 dan 66.249.66.91, yang diketahui merupakan alamat IP milik Google dan digunakan oleh layanan web crawler miliknya, seperti Googlebot. Selain dari Googlebot, IP terbesar selain crawler merupakan IP dari ISP yang terletak dari Iran. Yang menunjukkan bahwa website tersebut ditujukan untuk penduduk daerah sekitar Iran



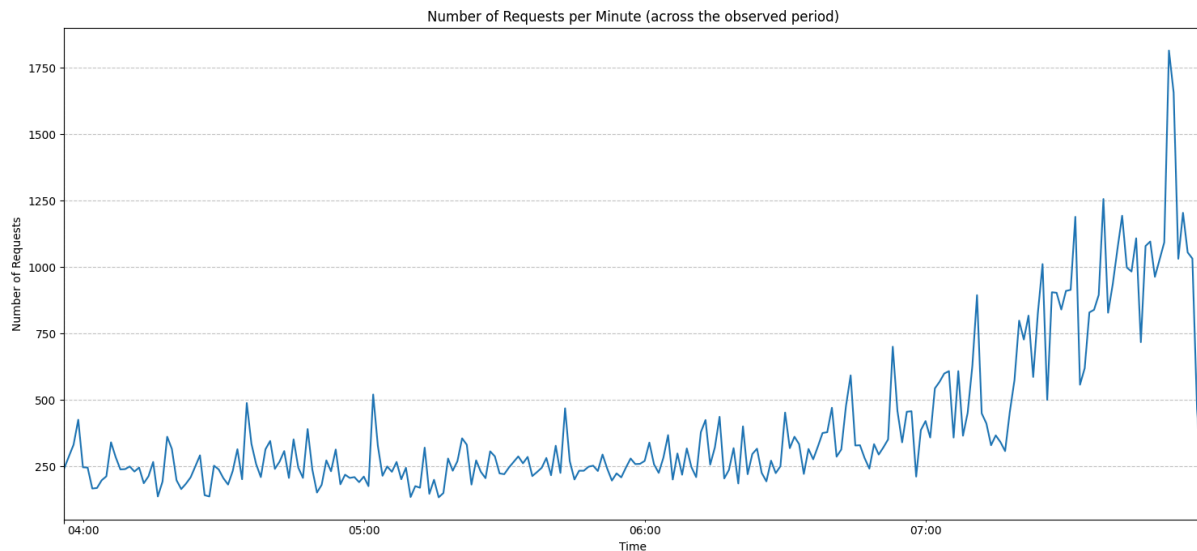
Berdasarkan visualisasi pada Gambar diatas, dapat dilihat bahwa kedua IP tersebut menyumbang persentase tertinggi dari total request, yaitu sekitar 31,4% dan 20,9%. Hal ini menunjukkan bahwa lebih dari setengah aktivitas yang tercatat pada sistem berasal dari proses crawling otomatis, bukan dari interaksi langsung pengguna manusia. Aktivitas semacam ini umumnya bertujuan untuk melakukan pengindeksan konten agar dapat ditampilkan pada hasil pencarian mesin pencari.

Selain melakukan analisis terhadap IP, terdapat juga kita lihat path yang diakses terbanyak, data tersebut menunjukkan apa saja yang sering diakses oleh pengguna ataupun webcrawler tersebut. Data tersebut kita lihat pada gambar berikut



Berdasarkan gambar tersebut, request terbanyak adalah static file terutama gambar. Mayoritas tersebut merupakan gambar dikarenakan website tersebut merupakan sebuah e-commerce. Yang dimana gambar tersebut merupakan sebuah product display, selain image filter dan static merupakan kategori yang sering diakses, dimana filter merupakan bagian web yang membatasi atau memfilter request yang dimiliki pengguna, seperti membatasi data kategori item kalau di dalam website tersebut.

Setelah menganalisis jenis request yang berada pada data tersebut, dapat juga dilihat analisis request per waktu, sehingga dapat terlihat jelas waktu puncak dari hasil pencarian website tersebut, yang mungkin untuk kedepannya dapat digunakan sebagai sebuah referensi ketika ingin melakukan scaling



Berdasarkan grafik jumlah request per menit, terlihat adanya pola peningkatan aktivitas yang cukup jelas seiring berjalannya waktu. Pada awal periode pengamatan, jumlah request berada pada kisaran yang relatif stabil, yaitu sekitar 200–300 permintaan per menit. Kondisi ini menunjukkan aktivitas sistem yang masih normal dan belum terlalu padat.

Memasuki rentang waktu sekitar pukul 07.00 dan seterusnya, terlihat adanya peningkatan jumlah request yang cukup signifikan. Grafik menunjukkan lonjakan bertahap hingga mencapai lebih dari 1.800 request per menit pada titik tertinggi. Pola ini selaras dengan peningkatan aktivitas pengguna pada jam operasional, di mana lebih banyak pengguna mulai mengakses sistem secara bersamaan.

Kenaikan jumlah request yang terjadi secara bertahap dan konsisten ini mengindikasikan bahwa sistem mulai menerima beban yang lebih tinggi seiring bertambahnya jumlah pengguna aktif. Hal tersebut merupakan pola yang wajar dalam penggunaan aplikasi, terutama pada jam-jam sibuk, dan menunjukkan bahwa sistem digunakan secara intens pada periode tersebut.

BAB V

5. KESIMPULAN

Berdasarkan seluruh tahapan implementasi dan analisis yang telah dilakukan, dapat disimpulkan bahwa Apache Hadoop dengan pemrograman MapReduce merupakan solusi yang efektif untuk memproses serta menganalisis data web server access log dalam skala besar. Melalui mekanisme pemrosesan paralel dan terdistribusi, sistem mampu menangani dataset berukuran 3,5 GB dengan lebih dari 10 juta baris log secara efisien, dengan waktu eksekusi relatif singkat dan penggunaan sumber daya yang terkontrol.

Hasil implementasi menunjukkan bahwa proses MapReduce berhasil mengekstraksi dan mengagregasi data alamat IP secara akurat, menghasilkan sebanyak 258.606 IP Address unik beserta frekuensi aksesnya. Distribusi frekuensi akses yang dihasilkan memperlihatkan pola heavy-tailed, di mana sebagian kecil IP menghasilkan volume request yang sangat besar dibandingkan IP lainnya. Pola ini konsisten dengan karakteristik trafik web nyata dan mengindikasikan dominasi aktivitas otomatis seperti web crawler dan bot, khususnya dari layanan mesin pencari seperti Googlebot.

Selain itu, tahap *monitoring* dan verifikasi melalui YARN Web UI serta pemeriksaan output di HDFS membuktikan bahwa seluruh komponen sistem Hadoop berjalan dengan stabil dan tanpa kegagalan. Penggunaan Combiner juga terbukti meningkatkan efisiensi sistem dengan mengurangi jumlah data yang harus diproses pada tahap Reduce hingga sekitar 97%, sehingga mempercepat proses agregasi secara keseluruhan.

Daftar Pustaka

Dabbas, E. (2019). Web Server Access Logs. Kaggle.

<https://www.kaggle.com/datasets/eliasdabbas/web-server-access-logs>